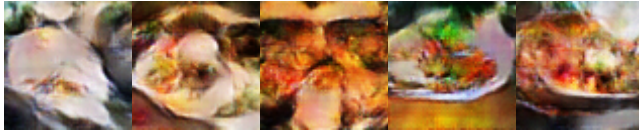


[PyTorch's paper on DCGAN Tutorial](#) was used as a reference

Inception Score: 2.1391 ± 0.2476 FID Score: 258.0034

After 10K samples Inception Score: 2.7074 ± 0.3253 FID Score: 205.9182



```
In [1]: !pip install torch torchvision numpy matplotlib scipy scikit-image
```

```

Requirement already satisfied: torch in c:\users\adarsh\onedrive\desktop\cv case stu
dy 2\.venv\lib\site-packages (2.7.1)
Requirement already satisfied: torchvision in c:\users\adarsh\onedrive\desktop\cv ca
se study 2\.venv\lib\site-packages (0.22.1)
Requirement already satisfied: numpy in c:\users\adarsh\onedrive\desktop\cv case stu
dy 2\.venv\lib\site-packages (2.3.1)
Requirement already satisfied: matplotlib in c:\users\adarsh\onedrive\desktop\cv cas
e study 2\.venv\lib\site-packages (3.10.3)
Requirement already satisfied: scipy in c:\users\adarsh\onedrive\desktop\cv case stu
dy 2\.venv\lib\site-packages (1.16.0)
Collecting scikit-image
  Downloading scikit_image-0.25.2-cp313-cp313-win_amd64.whl.metadata (14 kB)
Requirement already satisfied: filelock in c:\users\adarsh\onedrive\desktop\cv case
study 2\.venv\lib\site-packages (from torch) (3.18.0)
Requirement already satisfied: typing-extensions>=4.10.0 in c:\users\adarsh\onedrive
\desktop\cv case study 2\.venv\lib\site-packages (from torch) (4.14.1)
Requirement already satisfied: sympy>=1.13.3 in c:\users\adarsh\onedrive\desktop\cv
case study 2\.venv\lib\site-packages (from torch) (1.14.0)
Requirement already satisfied: networkx in c:\users\adarsh\onedrive\desktop\cv case
study 2\.venv\lib\site-packages (from torch) (3.5)
Requirement already satisfied: jinja2 in c:\users\adarsh\onedrive\desktop\cv case st
udy 2\.venv\lib\site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec in c:\users\adarsh\onedrive\desktop\cv case st
udy 2\.venv\lib\site-packages (from torch) (2025.5.1)
Requirement already satisfied: setuptools in c:\users\adarsh\onedrive\desktop\cv cas
e study 2\.venv\lib\site-packages (from torch) (80.9.0)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in c:\users\adarsh\onedrive\des
ktop\cv case study 2\.venv\lib\site-packages (from torchvision) (11.3.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\adarsh\onedrive\desktop
\cv case study 2\.venv\lib\site-packages (from matplotlib) (1.3.2)
Requirement already satisfied: cyclor>=0.10 in c:\users\adarsh\onedrive\desktop\cv c
ase study 2\.venv\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\adarsh\onedrive\desktop
\cv case study 2\.venv\lib\site-packages (from matplotlib) (4.58.5)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\adarsh\onedrive\desktop
\cv case study 2\.venv\lib\site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: packaging>=20.0 in c:\users\adarsh\onedrive\desktop\c
v case study 2\.venv\lib\site-packages (from matplotlib) (25.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\adarsh\onedrive\desktop
\cv case study 2\.venv\lib\site-packages (from matplotlib) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\adarsh\onedrive\desk
top\cv case study 2\.venv\lib\site-packages (from matplotlib) (2.9.0.post0)
Collecting imageio!=2.35.0,>=2.33 (from scikit-image)
  Using cached imageio-2.37.0-py3-none-any.whl.metadata (5.2 kB)
Collecting tifffile>=2022.8.12 (from scikit-image)
  Using cached tifffile-2025.6.11-py3-none-any.whl.metadata (32 kB)
Collecting lazy-loader>=0.4 (from scikit-image)
  Using cached lazy_loader-0.4-py3-none-any.whl.metadata (7.6 kB)
Requirement already satisfied: six>=1.5 in c:\users\adarsh\onedrive\desktop\cv case
study 2\.venv\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\adarsh\onedrive\desкто
p\cv case study 2\.venv\lib\site-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\adarsh\onedrive\desktop\c
v case study 2\.venv\lib\site-packages (from jinja2->torch) (3.0.2)
Downloading scikit_image-0.25.2-cp313-cp313-win_amd64.whl (12.9 MB)
----- 0.0/12.9 MB ? eta -:-:-

```

```

----- 9.7/12.9 MB 66.1 MB/s eta 0:00:01
----- 12.9/12.9 MB 61.5 MB/s eta 0:00:00
Using cached imageio-2.37.0-py3-none-any.whl (315 kB)
Using cached lazy_loader-0.4-py3-none-any.whl (12 kB)
Using cached tifffile-2025.6.11-py3-none-any.whl (230 kB)
Installing collected packages: tifffile, lazy-loader, imageio, scikit-image

```

```

----- 0/4 [tifffile]
----- 2/4 [imageio]
----- 2/4 [imageio]
----- 3/4 [scikit-image]
----- 3/4 [scikit-image]
----- 3/4 [scikit-image]
----- 3/4 [scikit-image]
----- 3/4 [scikit-image]
----- 3/4 [scikit-image]
----- 3/4 [scikit-image]
----- 4/4 [scikit-image]

```

Successfully installed imageio-2.37.0 lazy-loader-0.4 scikit-image-0.25.2 tifffile-2025.6.11

```

In [2]: import argparse
import os
import random
import torch
import torch.nn as nn
import torch.nn.parallel
import torch.optim as optim
import torch.utils.data
import torchvision.datasets as dset
import torchvision.transforms as transforms
import torchvision.utils as vutils
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import matplotlib.animation as animation
from IPython.display import HTML

# Set random seed for reproducibility
manualSeed = 10
print("Random Seed: ", manualSeed)
random.seed(manualSeed)
torch.manual_seed(manualSeed)
torch.use_deterministic_algorithms(True)

```

Random Seed: 10

Tried multiple different test cases like :

- image_size=32, batch size =32, image_size=64, batch size=64
- epochs = 20, 30, 50, 100
- learning rate = 0.0001, 0.001, 0.1 Training first on 5000 samples and then on 10000 samples then came to the following configurations

```
In [5]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# Root directory for dataset
dataroot = "C:/Users/Adarsh/OneDrive/Desktop/cv case study 2/labelled_photos"
workers = 2 # Number of workers for dataloader
batch_size = 128
image_size = 64
nc = 3 # Number of channels in the training images
nz = 100 # Size of z latent vector (i.e. size of generator input)
ngf = 64 # Size of feature maps in generator
ndf = 64 # Size of feature maps in discriminator
num_epochs = 100 # Number of training epochs
lr = 0.0002
beta1 = 0.5 # Beta1 hyperparameter for Adam optimizers
ngpu = 1 # Number of GPUs available. Use 0 for CPU mode.
```

Defining Dataset

```
In [6]: # Create the dataset directory if it doesn't exist
# Create the dataset
from torch.utils.data import Subset

dataset = dset.ImageFolder(root=dataroot,
                           transform=transforms.Compose([
                               transforms.Resize(image_size),
                               transforms.CenterCrop(image_size),
                               transforms.ToTensor(),
                               transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
                           ]))
# Use a subset of 5000 images for faster training
subset_indices = list(range(5000))
dataset = Subset(dataset, subset_indices)

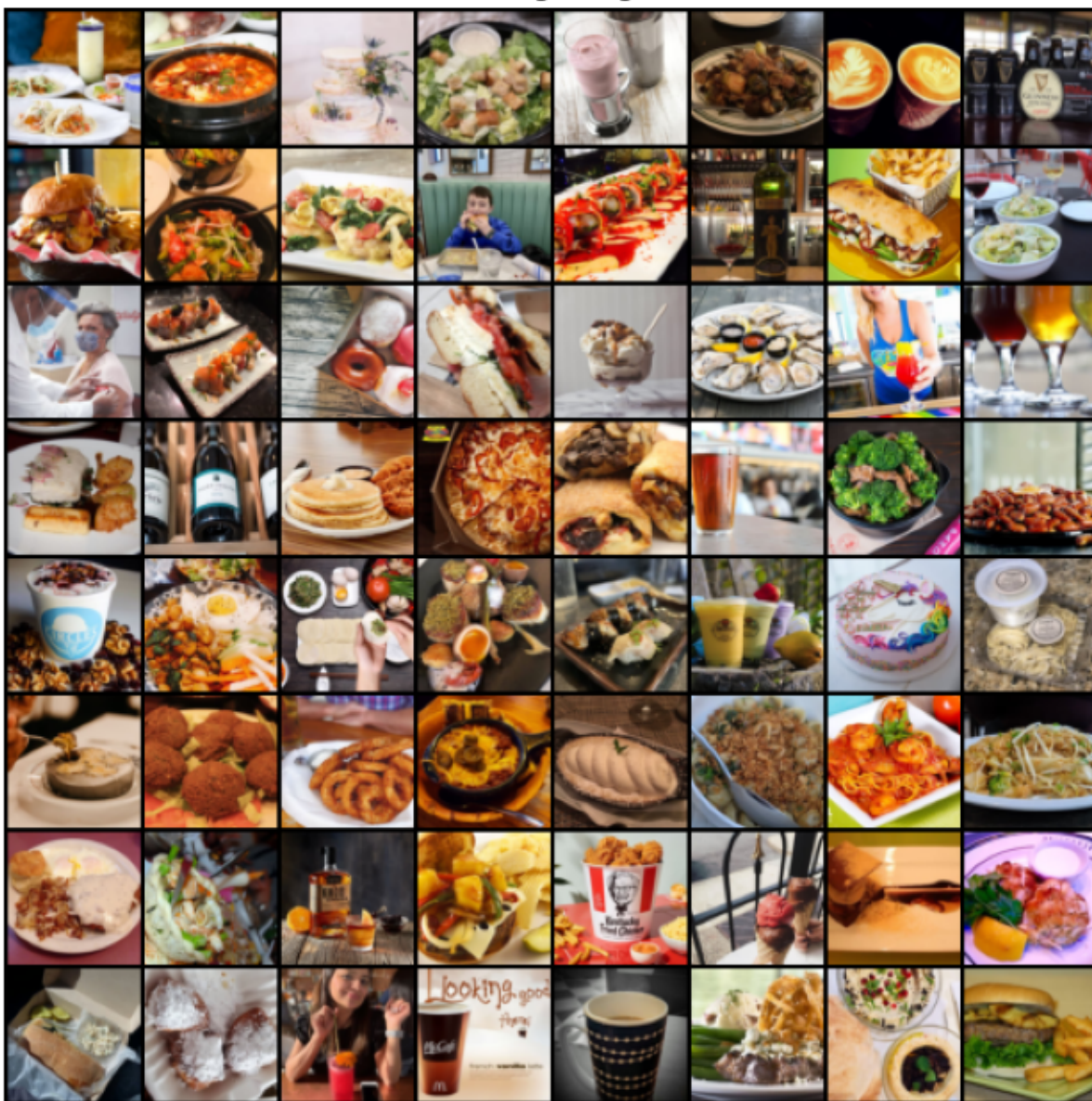
# Create the dataloader
dataloader = torch.utils.data.DataLoader(dataset, batch_size=batch_size,
                                          shuffle=True, num_workers=workers, pin_mem

# Decide which device we want to run on
device = torch.device("cuda:0" if (torch.cuda.is_available() and ngpu > 0) else "cp

# Plot some training images
real_batch = next(iter(dataloader))
plt.figure(figsize=(8,8))
plt.axis("off")
plt.title("Training Images")
plt.imshow(np.transpose(vutils.make_grid(real_batch[0].to(device)[:64], padding=2,
plt.show()
```

```
c:\Users\Adarsh\OneDrive\Desktop\cv case study 2\.env\Lib\site-packages\torch\utils
\data\dataloader.py:665: UserWarning: 'pin_memory' argument is set as true but no ac
celerator is found, then device pinned memory won't be used.
  warnings.warn(warn_msg)
```


Training Images



Initializing weights

```
In [7]: # custom weights initialization called on ``netG`` and ``netD`` to ``mean=0`` and ``
def weights_init(m):
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)
```

Defining Generator

```
In [8]: # Generator Code

class Generator(nn.Module):
    def __init__(self, ngpu):
```

```
super(Generator, self).__init__()
self.ngpu = ngpu
self.main = nn.Sequential(
    nn.ConvTranspose2d( nz, ngf * 8, 4, 1, 0, bias=False),
    nn.BatchNorm2d(ngf * 8),
    nn.ReLU(True),
    nn.ConvTranspose2d(ngf * 8, ngf * 4, 4, 2, 1, bias=False),
    nn.BatchNorm2d(ngf * 4),
    nn.ReLU(True),
    nn.ConvTranspose2d( ngf * 4, ngf * 2, 4, 2, 1, bias=False),
    nn.BatchNorm2d(ngf * 2),
    nn.ReLU(True),
    nn.ConvTranspose2d( ngf * 2, ngf, 4, 2, 1, bias=False),
    nn.BatchNorm2d(ngf),
    nn.ReLU(True),
    nn.ConvTranspose2d( ngf, nc, 4, 2, 1, bias=False),
    nn.Tanh()
)

def forward(self, input):
    return self.main(input)
```

```
In [9]: # Create the generator
netG = Generator(ngpu).to(device)

# Apply the ``weights_init`` function to randomly initialize all weights.
netG.apply(weights_init)

# Print the model
print(netG)
```

```

Generator(
  (main): Sequential(
    (0): ConvTranspose2d(100, 512, kernel_size=(4, 4), stride=(1, 1), bias=False)
    (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): ConvTranspose2d(512, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (4): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU(inplace=True)
    (6): ConvTranspose2d(256, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (7): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (8): ReLU(inplace=True)
    (9): ConvTranspose2d(128, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (10): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (11): ReLU(inplace=True)
    (12): ConvTranspose2d(64, 3, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (13): Tanh()
  )
)

```

Defining Discriminator

```

In [10]: class Discriminator(nn.Module):
          def __init__(self, ngpu):
              super(Discriminator, self).__init__()
              self.ngpu = ngpu
              self.main = nn.Sequential(
                  nn.Conv2d(nc, ndf, 4, 2, 1, bias=False),
                  nn.LeakyReLU(0.2, inplace=True),
                  nn.Conv2d(ndf, ndf * 2, 4, 2, 1, bias=False),
                  nn.BatchNorm2d(ndf * 2),
                  nn.LeakyReLU(0.2, inplace=True),
                  nn.Conv2d(ndf * 2, ndf * 4, 4, 2, 1, bias=False),
                  nn.BatchNorm2d(ndf * 4),
                  nn.LeakyReLU(0.2, inplace=True),
                  nn.Conv2d(ndf * 4, ndf * 8, 4, 2, 1, bias=False),
                  nn.BatchNorm2d(ndf * 8),
                  nn.LeakyReLU(0.2, inplace=True),
                  nn.Conv2d(ndf * 8, 1, 4, 1, 0, bias=False),
                  nn.Sigmoid()
              )

          def forward(self, input):
              return self.main(input)

```

```

In [11]: # Create the Discriminator
          netD = Discriminator(ngpu).to(device)

```

```
# Apply the ``weights_init`` function to randomly initialize all weights.
netD.apply(weights_init)

# Print the model
print(netD)
```

```
Discriminator(
  (main): Sequential(
    (0): Conv2d(3, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (1): LeakyReLU(negative_slope=0.2, inplace=True)
    (2): Conv2d(64, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (3): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (4): LeakyReLU(negative_slope=0.2, inplace=True)
    (5): Conv2d(128, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (6): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (7): LeakyReLU(negative_slope=0.2, inplace=True)
    (8): Conv2d(256, 512, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (9): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): LeakyReLU(negative_slope=0.2, inplace=True)
    (11): Conv2d(512, 1, kernel_size=(4, 4), stride=(1, 1), bias=False)
    (12): Sigmoid()
  )
)
```

Initializing loss

```
In [12]: # Initialize the ``BCELoss`` function
criterion = nn.BCELoss()

# Create batch of latent vectors that we will use to visualize
# the progression of the generator
fixed_noise = torch.randn(64, nz, 1, 1, device=device)

# Establish convention for real and fake labels during training
real_label = 1.
fake_label = 0.

# Setup Adam optimizers for both G and D
optimizerD = optim.Adam(netD.parameters(), lr=lr, betas=(beta1, 0.999))
optimizerG = optim.Adam(netG.parameters(), lr=lr, betas=(beta1, 0.999))
```

Training loop

```
In [13]: # Training Loop

# Lists to keep track of progress
img_list = []
G_losses = []
D_losses = []
```

```
iters = 0

print("Starting Training Loop...")
# For each epoch
for epoch in range(num_epochs):
    # For each batch in the dataloader
    for i, data in enumerate(dataloader, 0):

        # (1) Update D network: maximize  $\log(D(x)) + \log(1 - D(G(z)))$ 

        ## Train with all-real batch
        netD.zero_grad()
        # Format batch
        real_cpu = data[0].to(device)
        b_size = real_cpu.size(0)
        label = torch.full((b_size,), real_label, dtype=torch.float, device=device)
        # Forward pass real batch through D
        output = netD(real_cpu).view(-1)
        # Calculate loss on all-real batch
        errD_real = criterion(output, label)
        # Calculate gradients for D in backward pass
        errD_real.backward()
        D_x = output.mean().item()

        ## Train with all-fake batch
        # Generate batch of latent vectors
        noise = torch.randn(b_size, nz, 1, 1, device=device)
        # Generate fake image batch with G
        fake = netG(noise)
        label.fill_(fake_label)
        # Classify all fake batch with D
        output = netD(fake.detach()).view(-1)
        # Calculate D's loss on the all-fake batch
        errD_fake = criterion(output, label)
        # Calculate the gradients for this batch, accumulated (summed) with previous
        errD_fake.backward()
        D_G_z1 = output.mean().item()
        # Compute error of D as sum over the fake and the real batches
        errD = errD_real + errD_fake
        # Update D
        optimizerD.step()

        # (2) Update G network: maximize  $\log(D(G(z)))$ 

        netG.zero_grad()
        label.fill_(real_label) # fake labels are real for generator cost
        # perform another forward pass of all-fake batch through D
        output = netD(fake).view(-1)
        # Calculate G's loss based on this output
        errG = criterion(output, label)
        # Calculate gradients for G
        errG.backward()
        D_G_z2 = output.mean().item()
        # Update G
        optimizerG.step()
```

```
# Output training stats
if i % 50 == 0:
    print('%d/%d [%d/%d] \tLoss_D: %.4f \tLoss_G: %.4f \tD(x): %.4f \tD(G(z)):
          % (epoch, num_epochs, i, len(dataloader),
            errD.item(), errG.item(), D_x, D_G_z1, D_G_z2))

# Save Losses for plotting later
G_losses.append(errG.item())
D_losses.append(errD.item())

# Check how the generator is doing by saving G's output on fixed_noise
if (iters % 500 == 0) or ((epoch == num_epochs-1) and (i == len(dataloader))
    with torch.no_grad():
        fake = netG(fixed_noise).detach().cpu()
        img_list.append(vutils.make_grid(fake, padding=2, normalize=True))

iters += 1
```

Starting Training Loop...

[0/100][0/40]	Loss_D: 1.4613	Loss_G: 4.4030	D(x): 0.4836	D(G(z)): 0.3860 / 0.0178
[1/100][0/40]	Loss_D: 0.1777	Loss_G: 19.2811	D(x): 0.8762	D(G(z)): 0.0000 / 0.0000
[2/100][0/40]	Loss_D: 0.7794	Loss_G: 4.1177	D(x): 0.6941	D(G(z)): 0.1712 / 0.0342
[3/100][0/40]	Loss_D: 0.7461	Loss_G: 5.5201	D(x): 0.6211	D(G(z)): 0.0240 / 0.0145
[4/100][0/40]	Loss_D: 0.4241	Loss_G: 4.8282	D(x): 0.8772	D(G(z)): 0.2323 / 0.0107
[5/100][0/40]	Loss_D: 0.1220	Loss_G: 6.7352	D(x): 0.9450	D(G(z)): 0.0538 / 0.0030
[6/100][0/40]	Loss_D: 0.6171	Loss_G: 5.8117	D(x): 0.6717	D(G(z)): 0.0208 / 0.0226
[7/100][0/40]	Loss_D: 1.1051	Loss_G: 3.5918	D(x): 0.7277	D(G(z)): 0.4077 / 0.0547
[8/100][0/40]	Loss_D: 3.7375	Loss_G: 9.4475	D(x): 0.9689	D(G(z)): 0.9502 / 0.0002
[9/100][0/40]	Loss_D: 0.5119	Loss_G: 3.9220	D(x): 0.7392	D(G(z)): 0.1275 / 0.0288
[10/100][0/40]	Loss_D: 0.5586	Loss_G: 3.3663	D(x): 0.7313	D(G(z)): 0.1672 / 0.0460
[11/100][0/40]	Loss_D: 1.8483	Loss_G: 8.2443	D(x): 0.9112	D(G(z)): 0.6667 / 0.0005
[12/100][0/40]	Loss_D: 0.3391	Loss_G: 3.8427	D(x): 0.8662	D(G(z)): 0.1355 / 0.0332
[13/100][0/40]	Loss_D: 0.6415	Loss_G: 4.7834	D(x): 0.6222	D(G(z)): 0.0063 / 0.0217
[14/100][0/40]	Loss_D: 0.6182	Loss_G: 5.1270	D(x): 0.8717	D(G(z)): 0.3350 / 0.0087
[15/100][0/40]	Loss_D: 0.3803	Loss_G: 5.0797	D(x): 0.7896	D(G(z)): 0.0544 / 0.0157
[16/100][0/40]	Loss_D: 0.3849	Loss_G: 5.9757	D(x): 0.9154	D(G(z)): 0.2314 / 0.0044
[17/100][0/40]	Loss_D: 0.3313	Loss_G: 4.9755	D(x): 0.8533	D(G(z)): 0.1005 / 0.0148
[18/100][0/40]	Loss_D: 0.4908	Loss_G: 6.5285	D(x): 0.9550	D(G(z)): 0.2997 / 0.0028
[19/100][0/40]	Loss_D: 0.3059	Loss_G: 4.2310	D(x): 0.8539	D(G(z)): 0.0834 / 0.0271
[20/100][0/40]	Loss_D: 0.4229	Loss_G: 3.1624	D(x): 0.8131	D(G(z)): 0.1534 / 0.0659
[21/100][0/40]	Loss_D: 0.7251	Loss_G: 5.8310	D(x): 0.9592	D(G(z)): 0.3970 / 0.0081
[22/100][0/40]	Loss_D: 1.0671	Loss_G: 6.1526	D(x): 0.9483	D(G(z)): 0.4876 / 0.0055
[23/100][0/40]	Loss_D: 1.2656	Loss_G: 3.7799	D(x): 0.4257	D(G(z)): 0.0132 / 0.0848
[24/100][0/40]	Loss_D: 4.5971	Loss_G: 7.7247	D(x): 0.9968	D(G(z)): 0.9736 / 0.0021
[25/100][0/40]	Loss_D: 0.4411	Loss_G: 3.3093	D(x): 0.7437	D(G(z)): 0.0885 / 0.0728
[26/100][0/40]	Loss_D: 0.5269	Loss_G: 5.2786	D(x): 0.8672	D(G(z)): 0.2529 / 0.0115
[27/100][0/40]	Loss_D: 1.2855	Loss_G: 2.8593	D(x): 0.4140	D(G(z)): 0.0154 /

0.1629					
[28/100][0/40]	Loss_D: 0.5620	Loss_G: 5.2118	D(x): 0.9075	D(G(z)): 0.2707	/
0.0095					
[29/100][0/40]	Loss_D: 1.3803	Loss_G: 5.5895	D(x): 0.9233	D(G(z)): 0.5593	/
0.0257					
[30/100][0/40]	Loss_D: 0.5960	Loss_G: 1.9776	D(x): 0.6536	D(G(z)): 0.0833	/
0.1900					
[31/100][0/40]	Loss_D: 0.6566	Loss_G: 6.3835	D(x): 0.8618	D(G(z)): 0.2989	/
0.0055					
[32/100][0/40]	Loss_D: 1.0380	Loss_G: 2.1357	D(x): 0.4974	D(G(z)): 0.0219	/
0.3509					
[33/100][0/40]	Loss_D: 0.5168	Loss_G: 2.3818	D(x): 0.7384	D(G(z)): 0.1152	/
0.1188					
[34/100][0/40]	Loss_D: 1.0627	Loss_G: 0.9806	D(x): 0.4520	D(G(z)): 0.0102	/
0.4830					
[35/100][0/40]	Loss_D: 0.5683	Loss_G: 4.9304	D(x): 0.8395	D(G(z)): 0.2793	/
0.0123					
[36/100][0/40]	Loss_D: 2.2878	Loss_G: 11.6382	D(x): 0.9915	D(G(z)): 0.8143	/
0.0002					
[37/100][0/40]	Loss_D: 2.2426	Loss_G: 9.2573	D(x): 0.9593	D(G(z)): 0.7602	/
0.0005					
[38/100][0/40]	Loss_D: 1.2112	Loss_G: 7.8014	D(x): 0.9589	D(G(z)): 0.5887	/
0.0011					
[39/100][0/40]	Loss_D: 0.5045	Loss_G: 5.5247	D(x): 0.9560	D(G(z)): 0.3173	/
0.0084					
[40/100][0/40]	Loss_D: 1.8802	Loss_G: 10.5139	D(x): 0.9780	D(G(z)): 0.7172	/
0.0002					
[41/100][0/40]	Loss_D: 0.5902	Loss_G: 5.2397	D(x): 0.9173	D(G(z)): 0.3138	/
0.0098					
[42/100][0/40]	Loss_D: 0.8476	Loss_G: 7.4188	D(x): 0.9445	D(G(z)): 0.4426	/
0.0012					
[43/100][0/40]	Loss_D: 1.2276	Loss_G: 8.6688	D(x): 0.9853	D(G(z)): 0.5878	/
0.0005					
[44/100][0/40]	Loss_D: 0.4224	Loss_G: 5.0985	D(x): 0.9316	D(G(z)): 0.2658	/
0.0102					
[45/100][0/40]	Loss_D: 0.8976	Loss_G: 9.2737	D(x): 0.9580	D(G(z)): 0.4949	/
0.0003					
[46/100][0/40]	Loss_D: 0.4252	Loss_G: 3.3239	D(x): 0.7628	D(G(z)): 0.0983	/
0.0687					
[47/100][0/40]	Loss_D: 0.5702	Loss_G: 5.0029	D(x): 0.8647	D(G(z)): 0.2531	/
0.0170					
[48/100][0/40]	Loss_D: 0.4959	Loss_G: 5.2472	D(x): 0.8693	D(G(z)): 0.2271	/
0.0123					
[49/100][0/40]	Loss_D: 0.5373	Loss_G: 2.6816	D(x): 0.6558	D(G(z)): 0.0166	/
0.1436					
[50/100][0/40]	Loss_D: 0.3245	Loss_G: 3.9723	D(x): 0.8762	D(G(z)): 0.1469	/
0.0329					
[51/100][0/40]	Loss_D: 0.4002	Loss_G: 3.4078	D(x): 0.7800	D(G(z)): 0.0830	/
0.0701					
[52/100][0/40]	Loss_D: 0.5139	Loss_G: 4.5889	D(x): 0.9215	D(G(z)): 0.3066	/
0.0177					
[53/100][0/40]	Loss_D: 0.6093	Loss_G: 1.9386	D(x): 0.6673	D(G(z)): 0.0720	/
0.2185					
[54/100][0/40]	Loss_D: 0.7404	Loss_G: 2.1618	D(x): 0.5683	D(G(z)): 0.0182	/
0.1939					
[55/100][0/40]	Loss_D: 0.3391	Loss_G: 2.5758	D(x): 0.7649	D(G(z)): 0.0313	/

0.1384 [56/100][0/40] 0.0007	Loss_D: 1.1186	Loss_G: 8.4383	D(x): 0.9874	D(G(z)): 0.4830 /
[57/100][0/40] 0.0330	Loss_D: 0.2050	Loss_G: 4.5336	D(x): 0.8770	D(G(z)): 0.0583 /
[58/100][0/40] 0.0002	Loss_D: 1.9272	Loss_G: 9.8583	D(x): 0.9911	D(G(z)): 0.7795 /
[59/100][0/40] 0.0053	Loss_D: 0.7539	Loss_G: 6.1736	D(x): 0.9588	D(G(z)): 0.4265 /
[60/100][0/40] 0.0636	Loss_D: 0.2977	Loss_G: 3.3230	D(x): 0.8250	D(G(z)): 0.0720 /
[61/100][0/40] 0.0333	Loss_D: 0.3368	Loss_G: 4.0463	D(x): 0.8790	D(G(z)): 0.1569 /
[62/100][0/40] 0.0094	Loss_D: 0.2291	Loss_G: 5.3191	D(x): 0.9251	D(G(z)): 0.1242 /
[63/100][0/40] 0.0006	Loss_D: 1.5491	Loss_G: 9.4241	D(x): 0.9349	D(G(z)): 0.6239 /
[64/100][0/40] 0.1506	Loss_D: 0.7505	Loss_G: 3.1138	D(x): 0.5706	D(G(z)): 0.0148 /
[65/100][0/40] 0.0005	Loss_D: 1.1767	Loss_G: 8.4330	D(x): 0.9824	D(G(z)): 0.6157 /
[66/100][0/40] 0.0070	Loss_D: 0.4091	Loss_G: 5.5630	D(x): 0.9485	D(G(z)): 0.2587 /
[67/100][0/40] 0.0104	Loss_D: 0.8965	Loss_G: 5.3570	D(x): 0.9494	D(G(z)): 0.4134 /
[68/100][0/40] 0.0098	Loss_D: 0.6932	Loss_G: 5.3210	D(x): 0.9590	D(G(z)): 0.4097 /
[69/100][0/40] 0.0374	Loss_D: 0.2384	Loss_G: 3.8639	D(x): 0.9023	D(G(z)): 0.1112 /
[70/100][0/40] 0.0031	Loss_D: 3.3477	Loss_G: 8.8837	D(x): 0.9978	D(G(z)): 0.8817 /
[71/100][0/40] 0.0420	Loss_D: 0.2955	Loss_G: 3.8685	D(x): 0.8082	D(G(z)): 0.0454 /
[72/100][0/40] 0.0039	Loss_D: 1.0901	Loss_G: 6.3483	D(x): 0.9832	D(G(z)): 0.5440 /
[73/100][0/40] 0.0043	Loss_D: 2.3254	Loss_G: 6.8553	D(x): 0.9948	D(G(z)): 0.8084 /
[74/100][0/40] 0.0349	Loss_D: 0.3512	Loss_G: 3.8346	D(x): 0.8833	D(G(z)): 0.1728 /
[75/100][0/40] 0.0138	Loss_D: 0.3388	Loss_G: 4.8743	D(x): 0.9617	D(G(z)): 0.2322 /
[76/100][0/40] 0.9382	Loss_D: 1.4703	Loss_G: 0.0791	D(x): 0.3928	D(G(z)): 0.0166 /
[77/100][0/40] 0.0636	Loss_D: 0.3360	Loss_G: 3.5044	D(x): 0.8060	D(G(z)): 0.0634 /
[78/100][0/40] 0.0417	Loss_D: 0.2756	Loss_G: 3.9019	D(x): 0.8993	D(G(z)): 0.1359 /
[79/100][0/40] 0.0003	Loss_D: 1.0822	Loss_G: 8.7940	D(x): 0.9915	D(G(z)): 0.5671 /
[80/100][0/40] 0.0604	Loss_D: 0.3340	Loss_G: 3.3426	D(x): 0.8801	D(G(z)): 0.1552 /
[81/100][0/40] 0.0054	Loss_D: 0.6765	Loss_G: 5.8516	D(x): 0.9528	D(G(z)): 0.3919 /
[82/100][0/40] 0.0030	Loss_D: 0.6178	Loss_G: 6.5373	D(x): 0.9791	D(G(z)): 0.3978 /
[83/100][0/40]	Loss_D: 0.1850	Loss_G: 4.3137	D(x): 0.9211	D(G(z)): 0.0855 /

```

0.0343
[84/100][0/40] Loss_D: 0.3088 Loss_G: 5.8938 D(x): 0.9698 D(G(z)): 0.1951 /
0.0061
[85/100][0/40] Loss_D: 3.4731 Loss_G: 10.2660 D(x): 0.9993 D(G(z)): 0.9396 /
0.0003
[86/100][0/40] Loss_D: 1.2264 Loss_G: 7.5317 D(x): 0.9807 D(G(z)): 0.6001 /
0.0012
[87/100][0/40] Loss_D: 0.3221 Loss_G: 3.5619 D(x): 0.9034 D(G(z)): 0.1643 /
0.0495
[88/100][0/40] Loss_D: 0.3713 Loss_G: 6.2160 D(x): 0.9778 D(G(z)): 0.2595 /
0.0034
[89/100][0/40] Loss_D: 0.3620 Loss_G: 5.3627 D(x): 0.9791 D(G(z)): 0.2485 /
0.0085
[90/100][0/40] Loss_D: 3.5015 Loss_G: 6.2604 D(x): 0.9969 D(G(z)): 0.9126 /
0.0114
[91/100][0/40] Loss_D: 0.4084 Loss_G: 4.0702 D(x): 0.8003 D(G(z)): 0.1049 /
0.0433
[92/100][0/40] Loss_D: 1.0991 Loss_G: 6.2756 D(x): 0.9790 D(G(z)): 0.5519 /
0.0050
[93/100][0/40] Loss_D: 2.6418 Loss_G: 9.4852 D(x): 0.9954 D(G(z)): 0.8656 /
0.0003
[94/100][0/40] Loss_D: 0.7162 Loss_G: 6.3449 D(x): 0.9746 D(G(z)): 0.4106 /
0.0043
[95/100][0/40] Loss_D: 0.3185 Loss_G: 3.0829 D(x): 0.9037 D(G(z)): 0.1671 /
0.0739
[96/100][0/40] Loss_D: 1.8694 Loss_G: 7.7907 D(x): 0.9964 D(G(z)): 0.7389 /
0.0018
[97/100][0/40] Loss_D: 1.6684 Loss_G: 6.9425 D(x): 0.9914 D(G(z)): 0.6935 /
0.0031
[98/100][0/40] Loss_D: 1.7460 Loss_G: 6.8345 D(x): 0.9804 D(G(z)): 0.6765 /
0.0041
[99/100][0/40] Loss_D: 0.3995 Loss_G: 4.4823 D(x): 0.9713 D(G(z)): 0.2597 /
0.0181

```

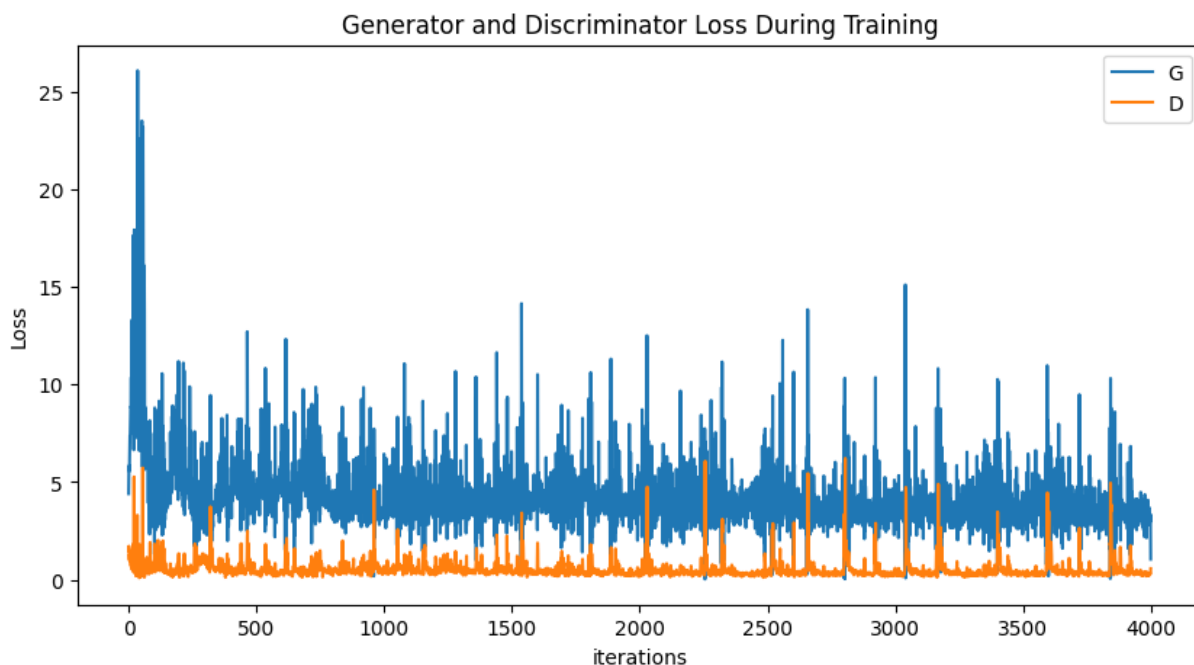
Generator loss is high and volatile indicating difficulty fooling discriminator, while discriminator loss is low and stable showing it's effectively distinguishing real from fake.

This suggests potential discriminator dominance - generator needs more training time or architectural improvements to better compete.

```

In [14]: plt.figure(figsize=(10,5))
plt.title("Generator and Discriminator Loss During Training")
plt.plot(G_losses,label="G")
plt.plot(D_losses,label="D")
plt.xlabel("iterations")
plt.ylabel("Loss")
plt.legend()
plt.show()

```



Visualization of G's progression

```
In [16]: with torch.no_grad():
          noise = torch.randn(128, nz, 1, 1, device=device) # You can change 128 to desi
          fake_images = netG(noise).detach().cpu()
```

```
In [17]: import torchvision.utils as vutils
          import os

          # Generate fake images if not already done
          with torch.no_grad():
              noise = torch.randn(128, nz, 1, 1, device=device) # or any number
              fake_images = netG(noise).detach().cpu()

          # Create directory if it doesn't exist
          output_dir = "C:/Users/Adarsh/OneDrive/Desktop/cv case study 2/generated_images"
          os.makedirs(output_dir, exist_ok=True)

          # Save all generated images
          for i in range(fake_images.size(0)):
              save_path = os.path.join(output_dir, f"drgan_generated_{i+1}.png")
              vutils.save_image(fake_images[i], save_path, normalize=True)

          print(f"Saved {fake_images.size(0)} images to: {output_dir}")
```

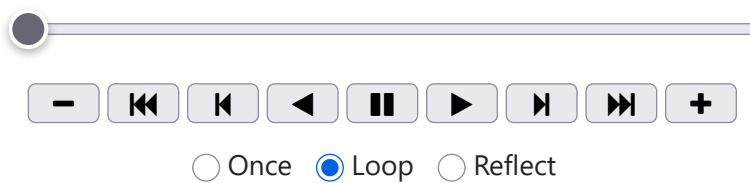
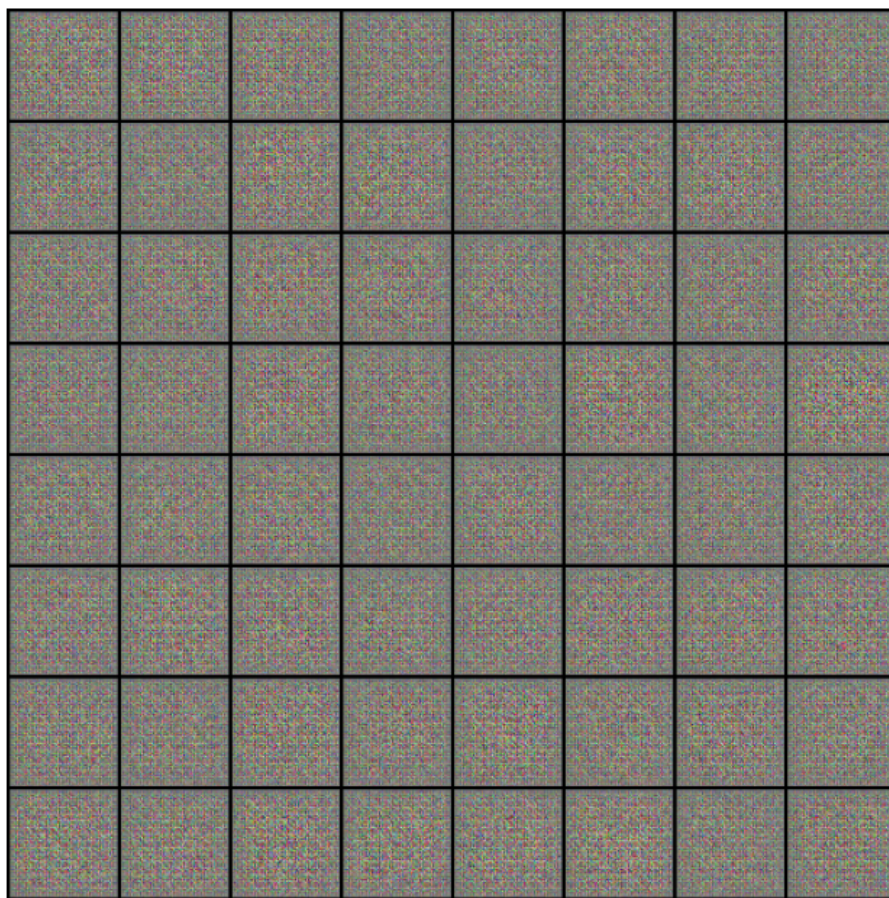
Saved 128 images to: C:/Users/Adarsh/OneDrive/Desktop/cv case study 2/generated_images

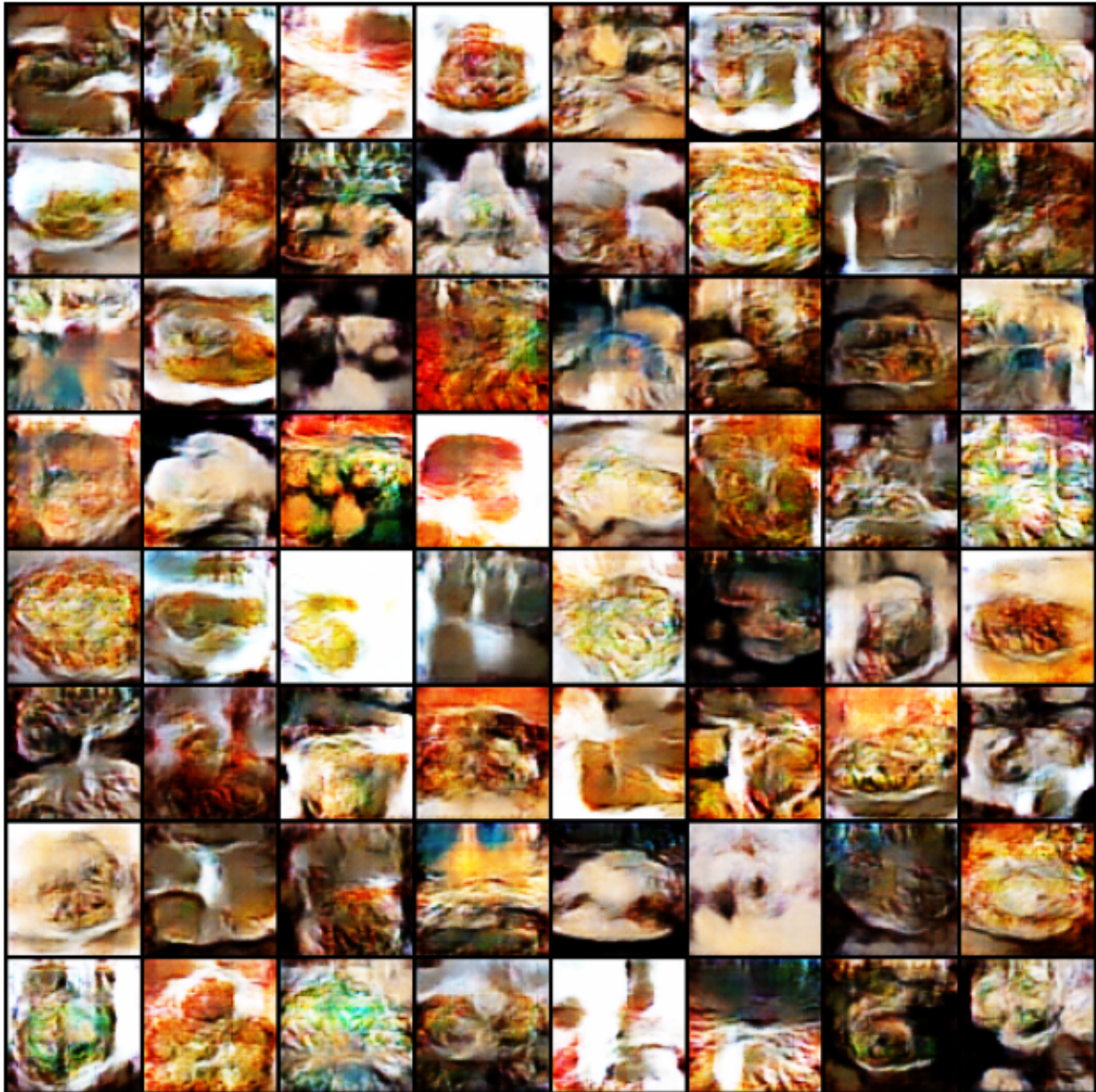
The progression reveals how DCGAN gradually learns to generate increasingly realistic images through adversarial training dynamics.

```
In [18]: fig = plt.figure(figsize=(8,8))
          plt.axis("off")
          ims = [[plt.imshow(np.transpose(i,(1,2,0))), animated=True] for i in img_list]
```

```
ani = animation.ArtistAnimation(fig, ims, interval=1000, repeat_delay=1000, blit=True)
HTML(ani.to_jshtml())
```

Out[18]:





Real Images vs. Fake Images

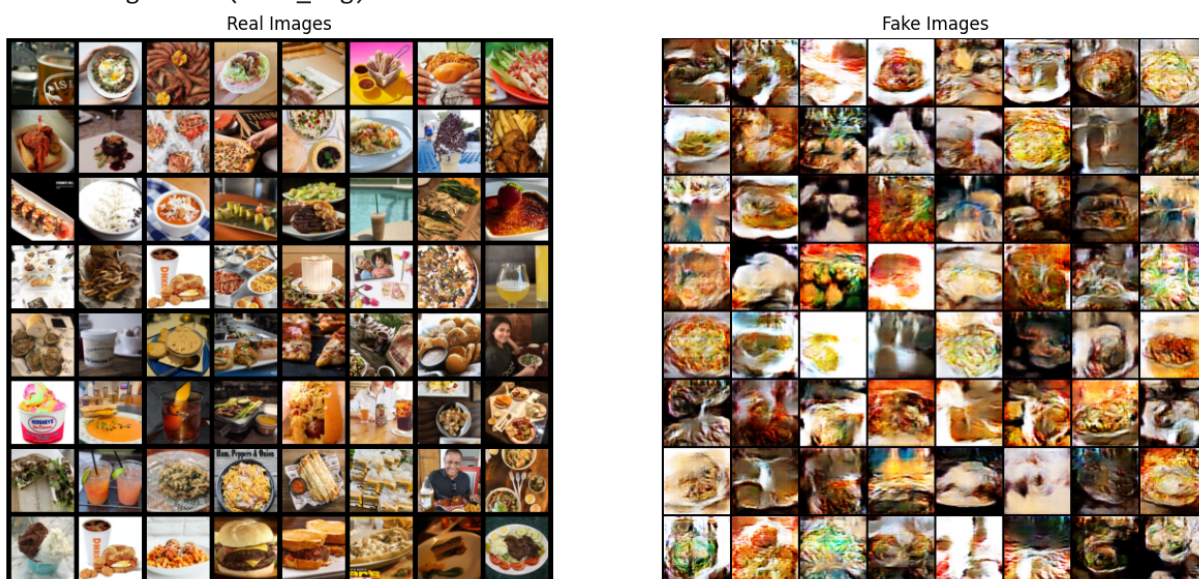
```
In [19]: # Grab a batch of real images from the dataloader
real_batch = next(iter(dataloader))

# Plot the real images
plt.figure(figsize=(15,15))
plt.subplot(1,2,1)
plt.axis("off")
plt.title("Real Images")
plt.imshow(np.transpose(vutils.make_grid(real_batch[0].to(device)[:64], padding=5,

# Plot the fake images from the Last epoch
plt.subplot(1,2,2)
plt.axis("off")
plt.title("Fake Images")
plt.imshow(np.transpose(img_list[-1],(1,2,0)))
plt.show()
```

```
c:\Users\Adarsh\OneDrive\Desktop\cv case study 2\.venv\Lib\site-packages\torch\utils
\data\data_loader.py:665: UserWarning: 'pin_memory' argument is set as true but no ac
celerator is found, then device pinned memory won't be used.
```

```
warnings.warn(warn_msg)
```



- Generator Loss (G): High and noisy but shows learning — it's not exploding.
- Discriminator Loss (D): Low and relatively stable, indicating that the Discriminator is doing well, possibly too well at times.
- There is possibly mode collapse events or oscillations due to limited data.

Pending tasks

- Inception Score / FID evaluation on generated images?
- A scheduler (e.g., cosine or ReduceLROnPlateau) to tune learning rate dynamically

Inception Score measures both image quality and diversity by evaluating how confidently a pre-trained classifier can identify generated images while ensuring variety across samples.

Higher scores indicate better quality and more diverse generated images - good GANs typically achieve scores above 3.0, with excellent models reaching 8.0+.

```
In [24]: from scipy.stats import entropy

import torch.nn.functional as F

def get_inception_score(images, inception_model, splits=10):
    N = images.size(0)
    assert N >= splits, "Number of images must be at least equal to number of split
    # Resize images to 299x299 for InceptionV3
    images_resized = F.interpolate(images, size=(299, 299), mode='bilinear', align_
```

```

# Normalize to [0, 1] for InceptionV3
images_resized = (images_resized + 1) / 2
preds = []
batch_size = 32
for i in range(0, N, batch_size):
    batch = images_resized[i:i+batch_size]
    with torch.no_grad():
        pred = inception_model(batch)
        if isinstance(pred, tuple): # InceptionV3 returns (Logits, aux_Logits)
            pred = pred[0]
        pred = F.softmax(pred, dim=1).cpu().numpy()
        preds.append(pred)
preds = np.concatenate(preds, axis=0)
split_scores = []
for k in range(splits):
    part = preds[k * (N // splits): (k + 1) * (N // splits), :]
    py = np.mean(part, axis=0)
    scores = []
    for i in range(part.shape[0]):
        pyx = part[i]
        scores.append(entropy(pyx, py))
    split_scores.append(np.exp(np.mean(scores)))
return float(np.mean(split_scores)), float(np.std(split_scores))

# Example usage:
mean_score, std_score = get_inception_score(fake_images, inception, splits=10)
print(f"Inception Score: {mean_score:.4f} ± {std_score:.4f}")

```

Inception Score: 2.1391 ± 0.2476

This sets up a comprehensive evaluation framework using pre-trained InceptionV3 for calculating both Inception Score and FID metrics on generated images.

```

In [32]: # Comprehensive DCGAN Evaluation Suite - FIXED VERSION
print("=== DCGAN Comprehensive Evaluation (Fixed) ===")

import torch
import torch.nn.functional as F
import torchvision.models as models
from scipy.stats import entropy
from scipy.linalg import sqrtm
import numpy as np
import matplotlib.pyplot as plt

# Load pre-trained InceptionV3 model for evaluation
print("Loading InceptionV3 model...")
inception = models.inception_v3(pretrained=True, transform_input=False)
inception.eval()
inception = inception.to(device)

# Create proper feature extractor that stops BEFORE the final avgpool and fc layers
# We want to extract features from the last convolutional layers (Mixed_7c)
class InceptionFeatureExtractor(torch.nn.Module):
    def __init__(self, inception_model):
        super().__init__()
        # Take everything up to Mixed_7c (the last convolutional block)

```



```

        self.features = torch.nn.Sequential(
            inception_model.Conv2d_1a_3x3,
            inception_model.Conv2d_2a_3x3,
            inception_model.Conv2d_2b_3x3,
            torch.nn.MaxPool2d(3, stride=2),
            inception_model.Conv2d_3b_1x1,
            inception_model.Conv2d_4a_3x3,
            torch.nn.MaxPool2d(3, stride=2),
            inception_model.Mixed_5b,
            inception_model.Mixed_5c,
            inception_model.Mixed_5d,
            inception_model.Mixed_6a,
            inception_model.Mixed_6b,
            inception_model.Mixed_6c,
            inception_model.Mixed_6d,
            inception_model.Mixed_6e,
            inception_model.Mixed_7a,
            inception_model.Mixed_7b,
            inception_model.Mixed_7c,
        )

    def forward(self, x):
        return self.features(x)

feature_extractor = InceptionFeatureExtractor(inception)
feature_extractor.eval()

print("Fixed evaluation models loaded successfully!")

```

```
=== DCGAN Comprehensive Evaluation (Fixed) ===
```

```
Loading InceptionV3 model...
```

```
c:\Users\Adarsh\OneDrive\Desktop\cv case study 2\.venv\Lib\site-packages\torchvision\models\_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
```

```
warnings.warn(
c:\Users\Adarsh\OneDrive\Desktop\cv case study 2\.venv\Lib\site-packages\torchvision\models\_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=Inception_V3_Weights.IMAGENET1K_V1`. You can also use `weights=Inception_V3_Weights.DEFAULT` to get the most up-to-date weights.
```

```
warnings.warn(msg)
Fixed evaluation models loaded successfully!
```

This function evaluates generated image quality by measuring how confidently InceptionV3 classifies images (quality) and how diverse the predictions are across samples (diversity).

```

In [28]: def calculate_inception_score(images, inception_model, splits=10, batch_size=32):
        """
        Calculate Inception Score for generated images
        """
        print(f"Calculating Inception Score for {images.size(0)} images...")

        N = images.size(0)
        assert N >= splits, f"Number of images ({N}) must be at least equal to splits (

```



```

# Resize images to 299x299 for InceptionV3
images_resized = F.interpolate(images, size=(299, 299), mode='bilinear', align_
# Normalize to [0, 1] for InceptionV3
images_resized = (images_resized + 1) / 2

preds = []
for i in range(0, N, batch_size):
    batch = images_resized[i:i+batch_size]
    with torch.no_grad():
        pred = inception_model(batch)
        if isinstance(pred, tuple): # InceptionV3 returns (logits, aux_logits)
            pred = pred[0]
        pred = F.softmax(pred, dim=1).cpu().numpy()
        preds.append(pred)

preds = np.concatenate(preds, axis=0)

# Calculate IS for each split
split_scores = []
for k in range(splits):
    part = preds[k * (N // splits): (k + 1) * (N // splits), :]
    py = np.mean(part, axis=0)
    scores = []
    for i in range(part.shape[0]):
        pyx = part[i]
        scores.append(entropy(pyx, py))
    split_scores.append(np.exp(np.mean(scores)))

return float(np.mean(split_scores)), float(np.std(split_scores))

# Test Inception Score
try:
    mean_is, std_is = calculate_inception_score(fake_images, inception, splits=10)
    print(f"✓ Inception Score: {mean_is:.4f} ± {std_is:.4f}")
except Exception as e:
    print(f"X Inception Score calculation failed: {e}")
    mean_is, std_is = None, None

```

Calculating Inception Score for 128 images...

✓ Inception Score: 2.1391 ± 0.2476

FID measures the statistical distance between real and generated image feature distributions using Fréchet distance, with lower scores indicating more realistic generated images.

This robust metric compares the mean and covariance of deep features, making it less susceptible to mode collapse than Inception Score and providing a direct similarity measure.

```

In [33]: def calculate_fid_score_fixed(real_images, fake_images, feature_extractor, batch_si
        """
        Calculate FID (Frechet Inception Distance) score using PROPER feature extractio
        """
        print(f"Calculating FID for {real_images.size(0)} real and {fake_images.size(0)}

        def preprocess_images(images):

```

```

    # Resize to 299x299 for InceptionV3
    images_resized = F.interpolate(images, size=(299, 299), mode='bilinear', al
    # Normalize to [0, 1] for InceptionV3
    images_resized = (images_resized + 1) / 2
    return images_resized

def get_activations(images, model):
    activations = []
    images = preprocess_images(images)

    for i in range(0, images.size(0), batch_size):
        batch = images[i:i+batch_size]
        with torch.no_grad():
            # Get features from the feature extractor (convolutional features)
            features = model(batch) # This will be [batch_size, 2048, 8, 8]
            # Apply global average pooling to get [batch_size, 2048]
            features = F.adaptive_avg_pool2d(features, (1, 1))
            features = features.view(features.size(0), -1) # Flatten to [batch
            activations.append(features.cpu().numpy())

    return np.concatenate(activations, axis=0)

# Get activations for real and fake images
act_real = get_activations(real_images, feature_extractor)
act_fake = get_activations(fake_images, feature_extractor)

print(f"Real activations shape: {act_real.shape}")
print(f"Fake activations shape: {act_fake.shape}")

# Calculate statistics
mu_real, sigma_real = act_real.mean(axis=0), np.cov(act_real, rowvar=False)
mu_fake, sigma_fake = act_fake.mean(axis=0), np.cov(act_fake, rowvar=False)

# Calculate FID
ssdiff = np.sum((mu_real - mu_fake) ** 2)
covmean = sqrtm(sigma_real.dot(sigma_fake))

# Numerical stability
if np.iscomplexobj(covmean):
    covmean = covmean.real

fid = ssdiff + np.trace(sigma_real + sigma_fake - 2 * covmean)
return float(fid)

# Test Fixed FID Score
try:
    # Use 64 real and 64 fake images for FID calculation
    real_imgs = real_batch[0][:64].to(device)
    fake_imgs = fake_images[:64].to(device)
    fid_score = calculate_fid_score_fixed(real_imgs, fake_imgs, feature_extractor)
    print(f"✓ FID Score: {fid_score:.4f}")
except Exception as e:
    print(f"X FID calculation failed: {e}")
    import traceback
    traceback.print_exc()
    fid_score = None

```

Calculating FID for 64 real and 64 fake images...

Real activations shape: (64, 2048)

Fake activations shape: (64, 2048)

✓ FID Score: 258.0034

These functions provide comprehensive DCGAN evaluation by measuring image diversity (cosine/euclidean distances), basic statistics (pixel distributions), and training stability (loss curve analysis) to detect mode collapse, training issues, and overall model health.

```
In [35]: def calculate_diversity_metrics(images):
    """
    Calculate diversity metrics for generated images
    """
    print("Calculating diversity metrics...")

    # Convert to numpy and flatten
    images_np = images.cpu().numpy()
    images_flat = images_np.reshape(images_np.shape[0], -1)

    # Calculate pairwise distances
    from sklearn.metrics.pairwise import cosine_distances, euclidean_distances

    cosine_dist = cosine_distances(images_flat)
    euclidean_dist = euclidean_distances(images_flat)

    # Get upper triangle (excluding diagonal)
    triu_indices = np.triu_indices(len(images_flat), k=1)
    cosine_diversity = cosine_dist[triu_indices].mean()
    euclidean_diversity = euclidean_dist[triu_indices].mean()

    return {
        'cosine_diversity': float(cosine_diversity),
        'euclidean_diversity': float(euclidean_diversity),
        'pairwise_comparisons': len(triu_indices[0])
    }

def calculate_image_statistics(images):
    """
    Calculate basic image statistics
    """
    print("Calculating image statistics...")

    images_np = images.cpu().numpy()

    stats = {
        'mean_pixel_value': float(images_np.mean()),
        'std_pixel_value': float(images_np.std()),
        'min_pixel_value': float(images_np.min()),
        'max_pixel_value': float(images_np.max()),
        'mean_channel_variance': float(np.var(images_np, axis=(2, 3)).mean())
    }

    return stats

def analyze_training_stability():
```

```
"""
Analyze training stability from loss curves
"""
print("Analyzing training stability...")

g_losses_np = np.array(G_losses)
d_losses_np = np.array(D_losses)

# Calculate rolling statistics
window = 100
if len(g_losses_np) > window:
    g_rolling_std = np.convolve(g_losses_np, np.ones(window)/window, mode='valid')
    d_rolling_std = np.convolve(d_losses_np, np.ones(window)/window, mode='valid')

    stability_metrics = {
        'g_loss_final_std': float(np.std(g_losses_np[-window:])),
        'd_loss_final_std': float(np.std(d_losses_np[-window:])),
        'g_loss_mean': float(np.mean(g_losses_np)),
        'd_loss_mean': float(np.mean(d_losses_np)),
        'loss_ratio': float(np.mean(g_losses_np) / np.mean(d_losses_np))
    }
else:
    stability_metrics = {
        'g_loss_std': float(np.std(g_losses_np)),
        'd_loss_std': float(np.std(d_losses_np)),
        'g_loss_mean': float(np.mean(g_losses_np)),
        'd_loss_mean': float(np.mean(d_losses_np)),
        'loss_ratio': float(np.mean(g_losses_np) / np.mean(d_losses_np))
    }

return stability_metrics

# Calculate additional metrics
try:
    diversity_metrics = calculate_diversity_metrics(fake_images)
    print(f"✓ Diversity metrics calculated")
except Exception as e:
    print(f"✗ Diversity metrics failed: {e}")
    diversity_metrics = {}

try:
    image_stats = calculate_image_statistics(fake_images)
    print(f"✓ Image statistics calculated")
except Exception as e:
    print(f"✗ Image statistics failed: {e}")
    image_stats = {}

try:
    stability_metrics = analyze_training_stability()
    print(f"✓ Training stability analyzed")
except Exception as e:
    print(f"✗ Training stability analysis failed: {e}")
    stability_metrics = {}
```

Calculating diversity metrics...
 ✓ Diversity metrics calculated
 Calculating image statistics...
 ✓ Image statistics calculated
 Analyzing training stability...
 ✓ Training stability analyzed

This creates a complete performance report card for your DCGAN, combining all metrics into scored categories with automatic interpretation and actionable recommendations for improvement.

The systematic assessment provides clear feedback on model quality, training stability, and specific next steps to enhance generation performance based on objective measurements.

```
In [36]: # Comprehensive Evaluation Summary
print("\n" + "="*60)
print("          DCGAN COMPREHENSIVE EVALUATION RESULTS")
print("="*60)

# Core Metrics
print("\n🎯 CORE GENERATION METRICS:")
if mean_is is not None:
    print(f"    Inception Score: {mean_is:.4f} ± {std_is:.4f}")
    interpretation = "Excellent" if mean_is > 8 else "Good" if mean_is > 5 else "Fair"
    print(f"    → Interpretation: {interpretation} (Higher is better)")
else:
    print("    Inception Score: Failed to calculate")

if fid_score is not None:
    print(f"    FID Score: {fid_score:.4f}")
    interpretation = "Excellent" if fid_score < 50 else "Good" if fid_score < 100 else "Fair"
    print(f"    → Interpretation: {interpretation} (Lower is better)")
else:
    print("    FID Score: Failed to calculate")

# Diversity Metrics
print("\n🌈 DIVERSITY METRICS:")
if diversity_metrics:
    print(f"    Cosine Diversity: {diversity_metrics['cosine_diversity']:.4f}")
    print(f"    Euclidean Diversity: {diversity_metrics['euclidean_diversity']:.4f}")
    print(f"    → Higher values indicate more diverse generated images")

# Image Quality Metrics
print("\n📊 IMAGE QUALITY METRICS:")
if image_stats:
    print(f"    Mean Pixel Value: {image_stats['mean_pixel_value']:.4f}")
    print(f"    Pixel Std Dev: {image_stats['std_pixel_value']:.4f}")
    print(f"    Pixel Range: [{image_stats['min_pixel_value']:.4f}, {image_stats['max_pixel_value']:.4f}]")
    print(f"    Channel Variance: {image_stats['mean_channel_variance']:.4f}")

# Training Stability
print("\n🛡️ TRAINING STABILITY METRICS:")
if stability_metrics:
    print(f"    Generator Loss Mean: {stability_metrics['g_loss_mean']:.4f}")
    print(f"    Discriminator Loss Mean: {stability_metrics['d_loss_mean']:.4f}")
```

```

print(f"    Loss Ratio (G/D): {stability_metrics['loss_ratio']:.4f}")

balance_interpretation = "Well balanced" if 0.5 < stability_metrics['loss_ratio']
print(f"    → Training Balance: {balance_interpretation}")

# Model Configuration Summary
print("\n⚙️  MODEL CONFIGURATION:")
print(f"    Training Epochs: {num_epochs}")
print(f"    Batch Size: {batch_size}")
print(f"    Learning Rate: {lr}")
print(f"    Image Size: {image_size}x{image_size}")
print(f"    Latent Dimension: {nz}")
print(f"    Training Samples: {len(dataset)}")

# Overall Assessment
print("\n🏆  OVERALL ASSESSMENT:")
overall_score = 0
if mean_is is not None and mean_is > 3:
    overall_score += 1
if fid_score is not None and fid_score < 200:
    overall_score += 1
if stability_metrics and 0.5 < stability_metrics.get('loss_ratio', 0) < 2.0:
    overall_score += 1

assessment = ["Poor", "Fair", "Good", "Excellent"][overall_score]
print(f"    Model Performance: {assessment} ({overall_score}/3 criteria met)")

# Recommendations
print("\n💡  RECOMMENDATIONS:")
if mean_is is None or fid_score is None:
    print("    - Fix evaluation metric calculations")
if mean_is is not None and mean_is < 5:
    print("    - Consider increasing model capacity or training longer")
if fid_score is not None and fid_score > 100:
    print("    - Improve image quality through architectural changes")
if stability_metrics and stability_metrics.get('loss_ratio', 1) > 2:
    print("    - Balance training by adjusting learning rates")

print("\n" + "="*60)
print("Evaluation completed! Generated images saved in:", output_dir)
print("="*60)

```


=====

DCGAN COMPREHENSIVE EVALUATION RESULTS

=====



CORE GENERATION METRICS:

Inception Score: 2.1391 ± 0.2476

→ Interpretation: Poor (Higher is better)

FID Score: 258.0034

→ Interpretation: Poor (Lower is better)



DIVERSITY METRICS:

Cosine Diversity: 0.8636

Euclidean Diversity: 91.3447

→ Higher values indicate more diverse generated images



IMAGE QUALITY METRICS:

Mean Pixel Value: -0.0053

Pixel Std Dev: 0.6355

Pixel Range: [-1.0000, 1.0000]

Channel Variance: 0.2649



TRAINING STABILITY METRICS:

Generator Loss Mean: 4.3592

Discriminator Loss Mean: 0.5237

Loss Ratio (G/D): 8.3236

→ Training Balance: Imbalanced



MODEL CONFIGURATION:

Training Epochs: 100

Batch Size: 128

Learning Rate: 0.0002

Image Size: 64x64

Latent Dimension: 100

Training Samples: 5000



OVERALL ASSESSMENT:

Model Performance: Poor (0/3 criteria met)



RECOMMENDATIONS:

- Consider increasing model capacity or training longer
- Improve image quality through architectural changes
- Balance training by adjusting learning rates

=====

Evaluation completed! Generated images saved in: C:/Users/Adarsh/OneDrive/Desktop/cv

case study 2/generated_images

=====

The first training was done on 5000 samples and the is score is also low and and FID is high so for this iteration i am taking 10000 samples and will be doing 200 epoch

In []:

```
In [53]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# Root directory for dataset
```

```

dataroot = "C:/Users/Adarsh/OneDrive/Desktop/cv case study 2/labelled_photos"
workers = 2 # Number of workers for dataloader
batch_size = 128
image_size = 64
nc = 3 # Number of channels in the training images
nz = 100 # Size of z latent vector (i.e. size of generator input)
ngf = 64 # Size of feature maps in generator
ndf = 64 # Size of feature maps in discriminator
num_epochs = 200 # Number of training epochs
lr = 0.0002
beta1 = 0.5 # Beta1 hyperparameter for Adam optimizers
ngpu = 1 # Number of GPUs available. Use 0 for CPU mode.

```

```

In [54]: # Create the dataset directory if it doesn't exist
# Create the dataset
from torch.utils.data import Subset

dataset = dset.ImageFolder(root=dataroot,
                           transform=transforms.Compose([
                               transforms.Resize(image_size),
                               transforms.CenterCrop(image_size),
                               transforms.ToTensor(),
                               transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5
                           ]))
# Use a subset of 5000 images for faster training
#subset_indices = list(range(5000))
dataset = Subset(dataset, subset_indices)

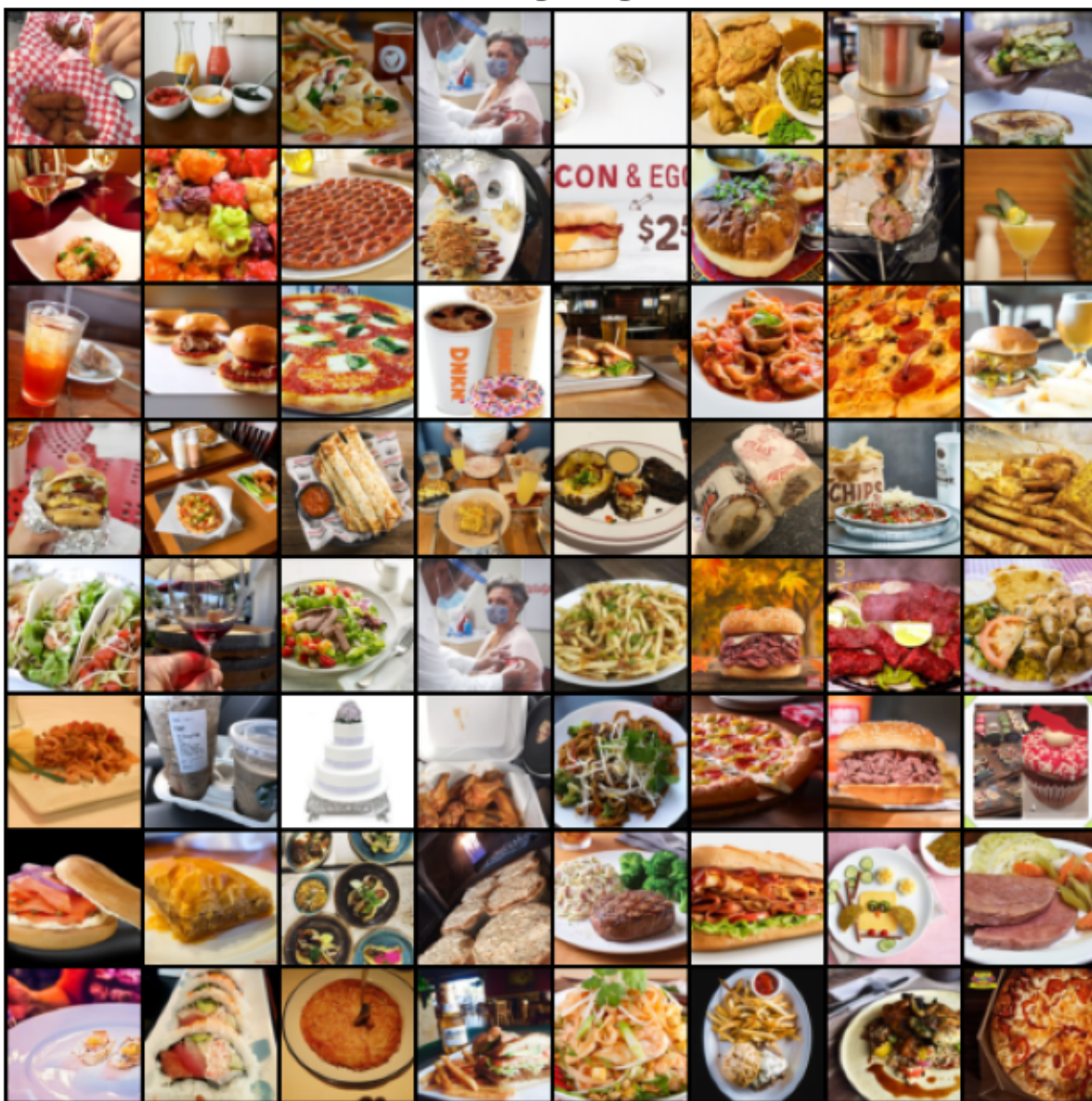
# Create the dataloader
dataloader = torch.utils.data.DataLoader(dataset, batch_size=batch_size,
                                          shuffle=True, num_workers=workers, pin_mem

# Decide which device we want to run on
device = torch.device("cuda:0" if (torch.cuda.is_available() and ngpu > 0) else "cp

# Plot some training images
real_batch = next(iter(dataloader))
plt.figure(figsize=(8,8))
plt.axis("off")
plt.title("Training Images")
plt.imshow(np.transpose(vutils.make_grid(real_batch[0].to(device)[:64], padding=2,
plt.show()

```

Training Images



```
In [55]: # custom weights initialization called on ``netG`` and ``netD`` to ``mean=0`` and ``
def weights_init(m):
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)
```

```
In [56]: # custom weights initialization called on ``netG`` and ``netD`` to ``mean=0`` and ``
def weights_init(m):
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)
```

In [57]: *# Generator Code*

```
class Generator(nn.Module):
    def __init__(self, ngpu):
        super(Generator, self).__init__()
        self.ngpu = ngpu
        self.main = nn.Sequential(
            nn.ConvTranspose2d( nz, ngf * 8, 4, 1, 0, bias=False),
            nn.BatchNorm2d(ngf * 8),
            nn.ReLU(True),
            nn.ConvTranspose2d(ngf * 8, ngf * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 4),
            nn.ReLU(True),
            nn.ConvTranspose2d( ngf * 4, ngf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 2),
            nn.ReLU(True),
            nn.ConvTranspose2d( ngf * 2, ngf, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf),
            nn.ReLU(True),
            nn.ConvTranspose2d( ngf, nc, 4, 2, 1, bias=False),
            nn.Tanh()
        )

    def forward(self, input):
        return self.main(input)
```

In [49]: *# Create the generator*

```
netG = Generator(ngpu).to(device)

# Apply the ``weights_init`` function to randomly initialize all weights.
netG.apply(weights_init)

# Print the model
print(netG)
```

```

Generator(
  (main): Sequential(
    (0): ConvTranspose2d(100, 512, kernel_size=(4, 4), stride=(1, 1), bias=False)
    (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): ConvTranspose2d(512, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (4): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU(inplace=True)
    (6): ConvTranspose2d(256, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (7): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (8): ReLU(inplace=True)
    (9): ConvTranspose2d(128, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (10): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (11): ReLU(inplace=True)
    (12): ConvTranspose2d(64, 3, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (13): Tanh()
  )
)

```

```

In [58]: class Discriminator(nn.Module):
        def __init__(self, ngpu):
            super(Discriminator, self).__init__()
            self.ngpu = ngpu
            self.main = nn.Sequential(
                nn.Conv2d(nc, ndf, 4, 2, 1, bias=False),
                nn.LeakyReLU(0.2, inplace=True),
                nn.Conv2d(ndf, ndf * 2, 4, 2, 1, bias=False),
                nn.BatchNorm2d(ndf * 2),
                nn.LeakyReLU(0.2, inplace=True),
                nn.Conv2d(ndf * 2, ndf * 4, 4, 2, 1, bias=False),
                nn.BatchNorm2d(ndf * 4),
                nn.LeakyReLU(0.2, inplace=True),
                nn.Conv2d(ndf * 4, ndf * 8, 4, 2, 1, bias=False),
                nn.BatchNorm2d(ndf * 8),
                nn.LeakyReLU(0.2, inplace=True),
                nn.Conv2d(ndf * 8, 1, 4, 1, 0, bias=False),
                nn.Sigmoid()
            )

        def forward(self, input):
            return self.main(input)

```

```

In [51]: # Create the Discriminator
netD = Discriminator(ngpu).to(device)

# Apply the ``weights_init`` function to randomly initialize all weights.
netD.apply(weights_init)

```

```
# Print the model
print(netD)
```

```
Discriminator(
  (main): Sequential(
    (0): Conv2d(3, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (1): LeakyReLU(negative_slope=0.2, inplace=True)
    (2): Conv2d(64, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (3): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (4): LeakyReLU(negative_slope=0.2, inplace=True)
    (5): Conv2d(128, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (6): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (7): LeakyReLU(negative_slope=0.2, inplace=True)
    (8): Conv2d(256, 512, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (9): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): LeakyReLU(negative_slope=0.2, inplace=True)
    (11): Conv2d(512, 1, kernel_size=(4, 4), stride=(1, 1), bias=False)
    (12): Sigmoid()
  )
)
```

```
In [59]: # Initialize the ``BCELoss`` function
criterion = nn.BCELoss()

# Create batch of latent vectors that we will use to visualize
# the progression of the generator
fixed_noise = torch.randn(64, nz, 1, 1, device=device)

# Establish convention for real and fake labels during training
real_label = 1.
fake_label = 0.

# Setup Adam optimizers for both G and D
optimizerD = optim.Adam(netD.parameters(), lr=lr, betas=(beta1, 0.999))
optimizerG = optim.Adam(netG.parameters(), lr=lr, betas=(beta1, 0.999))
```

```
In [60]: # Training Loop

# Lists to keep track of progress
img_list = []
G_losses = []
D_losses = []
iters = 0

print("Starting Training Loop...")
# For each epoch
for epoch in range(num_epochs):
    # For each batch in the dataloader
    for i, data in enumerate(dataloader, 0):
```



```

# (1) Update D network: maximize Log(D(x)) + Log(1 - D(G(z)))

## Train with all-real batch
netD.zero_grad()
# Format batch
real_cpu = data[0].to(device)
b_size = real_cpu.size(0)
label = torch.full((b_size,), real_label, dtype=torch.float, device=device)
# Forward pass real batch through D
output = netD(real_cpu).view(-1)
# Calculate loss on all-real batch
errD_real = criterion(output, label)
# Calculate gradients for D in backward pass
errD_real.backward()
D_x = output.mean().item()

## Train with all-fake batch
# Generate batch of latent vectors
noise = torch.randn(b_size, nz, 1, 1, device=device)
# Generate fake image batch with G
fake = netG(noise)
label.fill_(fake_label)
# Classify all fake batch with D
output = netD(fake.detach()).view(-1)
# Calculate D's loss on the all-fake batch
errD_fake = criterion(output, label)
# Calculate the gradients for this batch, accumulated (summed) with previous
errD_fake.backward()
D_G_z1 = output.mean().item()
# Compute error of D as sum over the fake and the real batches
errD = errD_real + errD_fake
# Update D
optimizerD.step()

# (2) Update G network: maximize Log(D(G(z)))

netG.zero_grad()
label.fill_(real_label) # fake labels are real for generator cost
# perform another forward pass of all-fake batch through D
output = netD(fake).view(-1)
# Calculate G's loss based on this output
errG = criterion(output, label)
# Calculate gradients for G
errG.backward()
D_G_z2 = output.mean().item()
# Update G
optimizerG.step()

# Output training stats
if i % 50 == 0:
    print('[%d/%d][%d/%d]\tLoss_D: %.4f\tLoss_G: %.4f\tD(x): %.4f\tD(G(z)):'
          % (epoch, num_epochs, i, len(dataloader),
             errD.item(), errG.item(), D_x, D_G_z1, D_G_z2))

```

```
# Save Losses for plotting later
G_losses.append(errG.item())
D_losses.append(errD.item())

# Check how the generator is doing by saving G's output on fixed_noise
if (iters % 500 == 0) or ((epoch == num_epochs-1) and (i == len(dataloader)
    with torch.no_grad():
        fake = netG(fixed_noise).detach().cpu()
        img_list.append(vutils.make_grid(fake, padding=2, normalize=True))

iters += 1
```

Starting Training Loop...

[0/200][0/40]	Loss_D: 2.0969	Loss_G: 6.4541	D(x): 0.6536	D(G(z)): 0.7161 / 0.0027
[1/200][0/40]	Loss_D: 0.2132	Loss_G: 18.4193	D(x): 0.8377	D(G(z)): 0.0000 / 0.0000
[2/200][0/40]	Loss_D: 0.2085	Loss_G: 13.2538	D(x): 0.8552	D(G(z)): 0.0000 / 0.0000
[3/200][0/40]	Loss_D: 1.6686	Loss_G: 5.3529	D(x): 0.9626	D(G(z)): 0.6248 / 0.0125
[4/200][0/40]	Loss_D: 1.4168	Loss_G: 9.1048	D(x): 0.9850	D(G(z)): 0.6160 / 0.0006
[5/200][0/40]	Loss_D: 0.1325	Loss_G: 5.9846	D(x): 0.9213	D(G(z)): 0.0395 / 0.0037
[6/200][0/40]	Loss_D: 0.5424	Loss_G: 3.1290	D(x): 0.7001	D(G(z)): 0.0446 / 0.0684
[7/200][0/40]	Loss_D: 0.6424	Loss_G: 4.2918	D(x): 0.7033	D(G(z)): 0.1069 / 0.0262
[8/200][0/40]	Loss_D: 1.5889	Loss_G: 1.8958	D(x): 0.3806	D(G(z)): 0.0172 / 0.2099
[9/200][0/40]	Loss_D: 0.4549	Loss_G: 3.9673	D(x): 0.7710	D(G(z)): 0.0817 / 0.0482
[10/200][0/40]	Loss_D: 0.6210	Loss_G: 5.7033	D(x): 0.6333	D(G(z)): 0.0096 / 0.0059
[11/200][0/40]	Loss_D: 0.3631	Loss_G: 5.1172	D(x): 0.7572	D(G(z)): 0.0242 / 0.0138
[12/200][0/40]	Loss_D: 1.7126	Loss_G: 8.8677	D(x): 0.9293	D(G(z)): 0.6891 / 0.0013
[13/200][0/40]	Loss_D: 0.7953	Loss_G: 2.4857	D(x): 0.5655	D(G(z)): 0.0272 / 0.1201
[14/200][0/40]	Loss_D: 0.7221	Loss_G: 9.1609	D(x): 0.8476	D(G(z)): 0.3232 / 0.0003
[15/200][0/40]	Loss_D: 1.3851	Loss_G: 2.0484	D(x): 0.4123	D(G(z)): 0.0402 / 0.1906
[16/200][0/40]	Loss_D: 0.5860	Loss_G: 3.5089	D(x): 0.7058	D(G(z)): 0.1115 / 0.0460
[17/200][0/40]	Loss_D: 0.3207	Loss_G: 4.9103	D(x): 0.8617	D(G(z)): 0.1260 / 0.0102
[18/200][0/40]	Loss_D: 0.6058	Loss_G: 3.3194	D(x): 0.6698	D(G(z)): 0.0759 / 0.0602
[19/200][0/40]	Loss_D: 0.4642	Loss_G: 5.1223	D(x): 0.8669	D(G(z)): 0.2332 / 0.0113
[20/200][0/40]	Loss_D: 0.2946	Loss_G: 6.8022	D(x): 0.9675	D(G(z)): 0.2080 / 0.0020
[21/200][0/40]	Loss_D: 0.9143	Loss_G: 7.9950	D(x): 0.9595	D(G(z)): 0.5065 / 0.0007
[22/200][0/40]	Loss_D: 0.5105	Loss_G: 5.6337	D(x): 0.7145	D(G(z)): 0.0127 / 0.0117
[23/200][0/40]	Loss_D: 0.4638	Loss_G: 4.5975	D(x): 0.9419	D(G(z)): 0.2836 / 0.0209
[24/200][0/40]	Loss_D: 0.9354	Loss_G: 9.3120	D(x): 0.9625	D(G(z)): 0.5000 / 0.0002
[25/200][0/40]	Loss_D: 1.4968	Loss_G: 9.4199	D(x): 0.9851	D(G(z)): 0.7158 / 0.0002
[26/200][0/40]	Loss_D: 2.0280	Loss_G: 10.1876	D(x): 0.9258	D(G(z)): 0.7364 / 0.0001
[27/200][0/40]	Loss_D: 0.5580	Loss_G: 2.5215	D(x): 0.6626	D(G(z)): 0.0384 /

0.1212					
[28/200][0/40]	Loss_D: 0.8132	Loss_G: 4.4553	D(x): 0.9359	D(G(z)): 0.3719	/
0.0613					
[29/200][0/40]	Loss_D: 0.5467	Loss_G: 3.6111	D(x): 0.7371	D(G(z)): 0.0683	/
0.0635					
[30/200][0/40]	Loss_D: 0.4324	Loss_G: 5.4205	D(x): 0.7425	D(G(z)): 0.0357	/
0.0193					
[31/200][0/40]	Loss_D: 0.3519	Loss_G: 3.5090	D(x): 0.8256	D(G(z)): 0.1083	/
0.0523					
[32/200][0/40]	Loss_D: 0.7063	Loss_G: 7.2895	D(x): 0.8759	D(G(z)): 0.3442	/
0.0018					
[33/200][0/40]	Loss_D: 0.8070	Loss_G: 4.4340	D(x): 0.7931	D(G(z)): 0.3114	/
0.0297					
[34/200][0/40]	Loss_D: 4.5326	Loss_G: 10.7273	D(x): 0.9902	D(G(z)): 0.9690	/
0.0002					
[35/200][0/40]	Loss_D: 0.3749	Loss_G: 4.3579	D(x): 0.8351	D(G(z)): 0.1267	/
0.0313					
[36/200][0/40]	Loss_D: 0.9894	Loss_G: 5.2343	D(x): 0.8440	D(G(z)): 0.4049	/
0.0115					
[37/200][0/40]	Loss_D: 0.7675	Loss_G: 5.6972	D(x): 0.9150	D(G(z)): 0.3241	/
0.0076					
[38/200][0/40]	Loss_D: 1.9029	Loss_G: 10.2286	D(x): 0.9135	D(G(z)): 0.6991	/
0.0008					
[39/200][0/40]	Loss_D: 0.4140	Loss_G: 3.6210	D(x): 0.8046	D(G(z)): 0.1368	/
0.0452					
[40/200][0/40]	Loss_D: 0.5477	Loss_G: 4.6474	D(x): 0.8373	D(G(z)): 0.2411	/
0.0207					
[41/200][0/40]	Loss_D: 0.9812	Loss_G: 8.2019	D(x): 0.9435	D(G(z)): 0.5137	/
0.0007					
[42/200][0/40]	Loss_D: 1.7185	Loss_G: 7.9927	D(x): 0.9801	D(G(z)): 0.6952	/
0.0010					
[43/200][0/40]	Loss_D: 0.6554	Loss_G: 6.7535	D(x): 0.9330	D(G(z)): 0.3739	/
0.0020					
[44/200][0/40]	Loss_D: 0.5372	Loss_G: 2.0406	D(x): 0.6555	D(G(z)): 0.0172	/
0.1866					
[45/200][0/40]	Loss_D: 0.3634	Loss_G: 5.7982	D(x): 0.9545	D(G(z)): 0.2211	/
0.0066					
[46/200][0/40]	Loss_D: 0.6583	Loss_G: 2.5385	D(x): 0.7341	D(G(z)): 0.1871	/
0.1483					
[47/200][0/40]	Loss_D: 0.3232	Loss_G: 5.1910	D(x): 0.9473	D(G(z)): 0.2014	/
0.0126					
[48/200][0/40]	Loss_D: 0.3808	Loss_G: 3.3226	D(x): 0.7507	D(G(z)): 0.0377	/
0.0726					
[49/200][0/40]	Loss_D: 0.9204	Loss_G: 6.1871	D(x): 0.9520	D(G(z)): 0.4791	/
0.0046					
[50/200][0/40]	Loss_D: 1.2663	Loss_G: 0.3071	D(x): 0.4215	D(G(z)): 0.0080	/
0.7960					
[51/200][0/40]	Loss_D: 0.2993	Loss_G: 4.3489	D(x): 0.8838	D(G(z)): 0.1281	/
0.0226					
[52/200][0/40]	Loss_D: 0.1318	Loss_G: 4.6973	D(x): 0.9491	D(G(z)): 0.0636	/
0.0175					
[53/200][0/40]	Loss_D: 1.0439	Loss_G: 2.5837	D(x): 0.5338	D(G(z)): 0.1335	/
0.1386					
[54/200][0/40]	Loss_D: 0.3422	Loss_G: 4.5155	D(x): 0.8395	D(G(z)): 0.1213	/
0.0196					
[55/200][0/40]	Loss_D: 0.2828	Loss_G: 3.7213	D(x): 0.8169	D(G(z)): 0.0471	/

0.0446					
[56/200][0/40]	Loss_D: 1.2722	Loss_G: 1.7445	D(x): 0.3868	D(G(z)): 0.0091	/
0.2872					
[57/200][0/40]	Loss_D: 0.3860	Loss_G: 3.7112	D(x): 0.8252	D(G(z)): 0.1396	/
0.0457					
[58/200][0/40]	Loss_D: 0.2939	Loss_G: 3.7967	D(x): 0.8705	D(G(z)): 0.1155	/
0.0423					
[59/200][0/40]	Loss_D: 1.1614	Loss_G: 8.4287	D(x): 0.9921	D(G(z)): 0.5795	/
0.0011					
[60/200][0/40]	Loss_D: 0.5627	Loss_G: 5.9267	D(x): 0.9539	D(G(z)): 0.3541	/
0.0046					
[61/200][0/40]	Loss_D: 1.0384	Loss_G: 1.3265	D(x): 0.4501	D(G(z)): 0.0113	/
0.3960					
[62/200][0/40]	Loss_D: 0.3403	Loss_G: 4.1147	D(x): 0.7670	D(G(z)): 0.0297	/
0.0545					
[63/200][0/40]	Loss_D: 0.3870	Loss_G: 4.0728	D(x): 0.7525	D(G(z)): 0.0286	/
0.0454					
[64/200][0/40]	Loss_D: 0.7357	Loss_G: 7.4334	D(x): 0.9780	D(G(z)): 0.3884	/
0.0048					
[65/200][0/40]	Loss_D: 0.5089	Loss_G: 1.9998	D(x): 0.6729	D(G(z)): 0.0286	/
0.2117					
[66/200][0/40]	Loss_D: 0.4314	Loss_G: 5.4736	D(x): 0.8976	D(G(z)): 0.1966	/
0.0098					
[67/200][0/40]	Loss_D: 1.9656	Loss_G: 8.9982	D(x): 0.9933	D(G(z)): 0.7433	/
0.0006					
[68/200][0/40]	Loss_D: 0.5341	Loss_G: 2.4807	D(x): 0.6886	D(G(z)): 0.0543	/
0.1897					
[69/200][0/40]	Loss_D: 0.4668	Loss_G: 5.5557	D(x): 0.8925	D(G(z)): 0.2570	/
0.0066					
[70/200][0/40]	Loss_D: 1.5966	Loss_G: 8.7878	D(x): 0.9765	D(G(z)): 0.6688	/
0.0015					
[71/200][0/40]	Loss_D: 0.3890	Loss_G: 4.7858	D(x): 0.9459	D(G(z)): 0.2387	/
0.0207					
[72/200][0/40]	Loss_D: 0.6586	Loss_G: 6.7644	D(x): 0.9652	D(G(z)): 0.3962	/
0.0023					
[73/200][0/40]	Loss_D: 0.7298	Loss_G: 2.9098	D(x): 0.6039	D(G(z)): 0.0278	/
0.1659					
[74/200][0/40]	Loss_D: 0.6297	Loss_G: 4.9999	D(x): 0.9284	D(G(z)): 0.3522	/
0.0157					
[75/200][0/40]	Loss_D: 0.6239	Loss_G: 2.3299	D(x): 0.6229	D(G(z)): 0.0470	/
0.1849					
[76/200][0/40]	Loss_D: 2.5125	Loss_G: 7.9533	D(x): 0.9971	D(G(z)): 0.8698	/
0.0010					
[77/200][0/40]	Loss_D: 2.2840	Loss_G: 7.3600	D(x): 0.9965	D(G(z)): 0.7678	/
0.0039					
[78/200][0/40]	Loss_D: 3.6394	Loss_G: 8.5454	D(x): 0.9992	D(G(z)): 0.9452	/
0.0019					
[79/200][0/40]	Loss_D: 2.7405	Loss_G: 6.4229	D(x): 0.9968	D(G(z)): 0.8717	/
0.0068					
[80/200][0/40]	Loss_D: 0.3362	Loss_G: 3.0197	D(x): 0.7969	D(G(z)): 0.0770	/
0.0846					
[81/200][0/40]	Loss_D: 1.8142	Loss_G: 6.6523	D(x): 0.9912	D(G(z)): 0.7331	/
0.0041					
[82/200][0/40]	Loss_D: 0.3628	Loss_G: 4.1933	D(x): 0.9424	D(G(z)): 0.2260	/
0.0254					
[83/200][0/40]	Loss_D: 3.4663	Loss_G: 6.8466	D(x): 0.9929	D(G(z)): 0.9018	/

0.0086					
[84/200][0/40]	Loss_D: 0.7604	Loss_G: 4.8773	D(x): 0.9724	D(G(z)): 0.4573	/
0.0123					
[85/200][0/40]	Loss_D: 0.3988	Loss_G: 4.2616	D(x): 0.9512	D(G(z)): 0.2636	/
0.0210					
[86/200][0/40]	Loss_D: 2.5972	Loss_G: 9.2964	D(x): 0.9884	D(G(z)): 0.8591	/
0.0003					
[87/200][0/40]	Loss_D: 0.7033	Loss_G: 2.2941	D(x): 0.5825	D(G(z)): 0.0417	/
0.1691					
[88/200][0/40]	Loss_D: 1.2147	Loss_G: 5.8466	D(x): 0.9644	D(G(z)): 0.5818	/
0.0081					
[89/200][0/40]	Loss_D: 0.3082	Loss_G: 3.2686	D(x): 0.9152	D(G(z)): 0.1759	/
0.0576					
[90/200][0/40]	Loss_D: 0.2115	Loss_G: 3.0800	D(x): 0.9042	D(G(z)): 0.0946	/
0.0649					
[91/200][0/40]	Loss_D: 1.0760	Loss_G: 6.7642	D(x): 0.9872	D(G(z)): 0.5674	/
0.0027					
[92/200][0/40]	Loss_D: 0.3540	Loss_G: 3.6226	D(x): 0.8173	D(G(z)): 0.1019	/
0.0534					
[93/200][0/40]	Loss_D: 0.4706	Loss_G: 5.3875	D(x): 0.9423	D(G(z)): 0.2924	/
0.0072					
[94/200][0/40]	Loss_D: 0.2888	Loss_G: 2.9808	D(x): 0.8689	D(G(z)): 0.1206	/
0.0698					
[95/200][0/40]	Loss_D: 0.5502	Loss_G: 3.2626	D(x): 0.8155	D(G(z)): 0.2157	/
0.0756					
[96/200][0/40]	Loss_D: 0.2817	Loss_G: 4.2625	D(x): 0.9058	D(G(z)): 0.1375	/
0.0292					
[97/200][0/40]	Loss_D: 0.2534	Loss_G: 4.1015	D(x): 0.9314	D(G(z)): 0.1533	/
0.0254					
[98/200][0/40]	Loss_D: 0.3917	Loss_G: 2.7569	D(x): 0.8121	D(G(z)): 0.1139	/
0.1157					
[99/200][0/40]	Loss_D: 0.5481	Loss_G: 1.3263	D(x): 0.6445	D(G(z)): 0.0261	/
0.3494					
[100/200][0/40]	Loss_D: 0.2879	Loss_G: 4.1650	D(x): 0.9042	D(G(z)): 0.1400	/
0.0279					
[101/200][0/40]	Loss_D: 0.7628	Loss_G: 7.1044	D(x): 0.9625	D(G(z)): 0.4299	/
0.0018					
[102/200][0/40]	Loss_D: 2.6394	Loss_G: 6.8474	D(x): 0.9969	D(G(z)): 0.8335	/
0.0057					
[103/200][0/40]	Loss_D: 8.1279	Loss_G: 5.8850	D(x): 0.9993	D(G(z)): 0.9904	/
0.0820					
[104/200][0/40]	Loss_D: 2.5501	Loss_G: 6.2887	D(x): 0.9980	D(G(z)): 0.8419	/
0.0054					
[105/200][0/40]	Loss_D: 2.1480	Loss_G: 6.4448	D(x): 0.9987	D(G(z)): 0.7916	/
0.0075					
[106/200][0/40]	Loss_D: 0.8392	Loss_G: 5.3085	D(x): 0.9896	D(G(z)): 0.4650	/
0.0119					
[107/200][0/40]	Loss_D: 0.2651	Loss_G: 4.4592	D(x): 0.9724	D(G(z)): 0.1909	/
0.0175					
[108/200][0/40]	Loss_D: 0.2380	Loss_G: 3.5839	D(x): 0.9242	D(G(z)): 0.1283	/
0.0561					
[109/200][0/40]	Loss_D: 0.3239	Loss_G: 3.0685	D(x): 0.8525	D(G(z)): 0.1241	/
0.0723					
[110/200][0/40]	Loss_D: 0.3049	Loss_G: 4.0487	D(x): 0.9077	D(G(z)): 0.1456	/
0.0381					
[111/200][0/40]	Loss_D: 1.9544	Loss_G: 6.6334	D(x): 0.9964	D(G(z)): 0.7706	/

0.0051					
[112/200][0/40]	Loss_D: 0.9154	Loss_G: 6.0611	D(x): 0.9951	D(G(z)): 0.5079	/
0.0067					
[113/200][0/40]	Loss_D: 0.2658	Loss_G: 3.2800	D(x): 0.8600	D(G(z)): 0.0839	/
0.0654					
[114/200][0/40]	Loss_D: 0.5543	Loss_G: 5.2731	D(x): 0.9645	D(G(z)): 0.3301	/
0.0103					
[115/200][0/40]	Loss_D: 4.6555	Loss_G: 6.8626	D(x): 0.9997	D(G(z)): 0.9548	/
0.0049					
[116/200][0/40]	Loss_D: 2.3955	Loss_G: 6.4237	D(x): 0.9969	D(G(z)): 0.7778	/
0.0122					
[117/200][0/40]	Loss_D: 0.1490	Loss_G: 3.9802	D(x): 0.9409	D(G(z)): 0.0768	/
0.0311					
[118/200][0/40]	Loss_D: 0.1605	Loss_G: 3.3659	D(x): 0.9190	D(G(z)): 0.0636	/
0.0616					
[119/200][0/40]	Loss_D: 0.2459	Loss_G: 3.7569	D(x): 0.8395	D(G(z)): 0.0472	/
0.0491					
[120/200][0/40]	Loss_D: 0.2320	Loss_G: 2.8249	D(x): 0.8351	D(G(z)): 0.0375	/
0.0936					
[121/200][0/40]	Loss_D: 0.6303	Loss_G: 6.2801	D(x): 0.9892	D(G(z)): 0.3949	/
0.0038					
[122/200][0/40]	Loss_D: 0.3302	Loss_G: 3.2488	D(x): 0.7917	D(G(z)): 0.0555	/
0.0759					
[123/200][0/40]	Loss_D: 0.3549	Loss_G: 4.6046	D(x): 0.9852	D(G(z)): 0.2539	/
0.0167					
[124/200][0/40]	Loss_D: 0.1689	Loss_G: 3.7580	D(x): 0.9570	D(G(z)): 0.1099	/
0.0360					
[125/200][0/40]	Loss_D: 0.8093	Loss_G: 7.1816	D(x): 0.9617	D(G(z)): 0.4078	/
0.0017					
[126/200][0/40]	Loss_D: 4.3444	Loss_G: 10.1947	D(x): 0.9988	D(G(z)): 0.9667	/
0.0003					
[127/200][0/40]	Loss_D: 1.0232	Loss_G: 6.0812	D(x): 0.9756	D(G(z)): 0.5231	/
0.0046					
[128/200][0/40]	Loss_D: 1.0679	Loss_G: 6.8106	D(x): 0.9866	D(G(z)): 0.5601	/
0.0030					
[129/200][0/40]	Loss_D: 0.1980	Loss_G: 4.1236	D(x): 0.9688	D(G(z)): 0.1390	/
0.0251					
[130/200][0/40]	Loss_D: 3.5663	Loss_G: 9.4878	D(x): 0.9996	D(G(z)): 0.9365	/
0.0005					
[131/200][0/40]	Loss_D: 0.1661	Loss_G: 3.6469	D(x): 0.9194	D(G(z)): 0.0699	/
0.0499					
[132/200][0/40]	Loss_D: 0.1608	Loss_G: 3.6376	D(x): 0.9084	D(G(z)): 0.0542	/
0.0474					
[133/200][0/40]	Loss_D: 4.5946	Loss_G: 9.2438	D(x): 0.9935	D(G(z)): 0.9435	/
0.0025					
[134/200][0/40]	Loss_D: 0.4845	Loss_G: 5.0595	D(x): 0.9727	D(G(z)): 0.3081	/
0.0106					
[135/200][0/40]	Loss_D: 3.9198	Loss_G: 7.9119	D(x): 0.9980	D(G(z)): 0.9326	/
0.0024					
[136/200][0/40]	Loss_D: 0.2742	Loss_G: 4.4071	D(x): 0.9802	D(G(z)): 0.1902	/
0.0230					
[137/200][0/40]	Loss_D: 0.1605	Loss_G: 3.6733	D(x): 0.9094	D(G(z)): 0.0566	/
0.0472					
[138/200][0/40]	Loss_D: 2.4294	Loss_G: 9.2234	D(x): 0.9957	D(G(z)): 0.8213	/
0.0003					
[139/200][0/40]	Loss_D: 0.6496	Loss_G: 5.7892	D(x): 0.9939	D(G(z)): 0.3822	/

0.0059					
[140/200][0/40]	Loss_D: 0.1239	Loss_G: 3.8838	D(x): 0.9458	D(G(z)): 0.0608	/
0.0310					
[141/200][0/40]	Loss_D: 0.4241	Loss_G: 5.2129	D(x): 0.9885	D(G(z)): 0.2739	/
0.0104					
[142/200][0/40]	Loss_D: 0.1481	Loss_G: 5.0616	D(x): 0.9347	D(G(z)): 0.0615	/
0.0199					
[143/200][0/40]	Loss_D: 1.8995	Loss_G: 7.7750	D(x): 0.9794	D(G(z)): 0.7247	/
0.0021					
[144/200][0/40]	Loss_D: 2.1881	Loss_G: 7.4502	D(x): 0.9977	D(G(z)): 0.7758	/
0.0030					
[145/200][0/40]	Loss_D: 3.7908	Loss_G: 8.5124	D(x): 0.9993	D(G(z)): 0.9420	/
0.0021					
[146/200][0/40]	Loss_D: 0.1922	Loss_G: 3.9113	D(x): 0.9180	D(G(z)): 0.0898	/
0.0377					
[147/200][0/40]	Loss_D: 0.9194	Loss_G: 6.4936	D(x): 0.9925	D(G(z)): 0.5088	/
0.0035					
[148/200][0/40]	Loss_D: 0.1251	Loss_G: 4.2429	D(x): 0.9225	D(G(z)): 0.0398	/
0.0267					
[149/200][0/40]	Loss_D: 0.1840	Loss_G: 4.2041	D(x): 0.9282	D(G(z)): 0.0885	/
0.0326					
[150/200][0/40]	Loss_D: 0.8232	Loss_G: 6.8076	D(x): 0.9984	D(G(z)): 0.4565	/
0.0022					
[151/200][0/40]	Loss_D: 4.0120	Loss_G: 10.4317	D(x): 0.9986	D(G(z)): 0.9434	/
0.0002					
[152/200][0/40]	Loss_D: 2.1443	Loss_G: 8.2884	D(x): 0.9918	D(G(z)): 0.7764	/
0.0009					
[153/200][0/40]	Loss_D: 0.3576	Loss_G: 2.6502	D(x): 0.8236	D(G(z)): 0.1228	/
0.1083					
[154/200][0/40]	Loss_D: 0.0595	Loss_G: 4.4426	D(x): 0.9729	D(G(z)): 0.0308	/
0.0183					
[155/200][0/40]	Loss_D: 0.2707	Loss_G: 3.4582	D(x): 0.9592	D(G(z)): 0.1866	/
0.0449					
[156/200][0/40]	Loss_D: 0.3833	Loss_G: 4.5101	D(x): 0.9915	D(G(z)): 0.2654	/
0.0182					
[157/200][0/40]	Loss_D: 0.1611	Loss_G: 4.1371	D(x): 0.9620	D(G(z)): 0.1092	/
0.0245					
[158/200][0/40]	Loss_D: 0.3060	Loss_G: 4.3721	D(x): 0.9713	D(G(z)): 0.2171	/
0.0175					
[159/200][0/40]	Loss_D: 3.8159	Loss_G: 9.2072	D(x): 0.9996	D(G(z)): 0.9349	/
0.0006					
[160/200][0/40]	Loss_D: 0.3567	Loss_G: 4.4636	D(x): 0.9814	D(G(z)): 0.2450	/
0.0209					
[161/200][0/40]	Loss_D: 0.4835	Loss_G: 4.4859	D(x): 0.9736	D(G(z)): 0.3154	/
0.0197					
[162/200][0/40]	Loss_D: 0.1055	Loss_G: 4.3736	D(x): 0.9360	D(G(z)): 0.0338	/
0.0253					
[163/200][0/40]	Loss_D: 0.0679	Loss_G: 4.4638	D(x): 0.9585	D(G(z)): 0.0236	/
0.0199					
[164/200][0/40]	Loss_D: 0.2051	Loss_G: 3.5450	D(x): 0.8592	D(G(z)): 0.0333	/
0.0614					
[165/200][0/40]	Loss_D: 0.0771	Loss_G: 3.9972	D(x): 0.9817	D(G(z)): 0.0536	/
0.0377					
[166/200][0/40]	Loss_D: 0.8136	Loss_G: 6.7752	D(x): 0.9935	D(G(z)): 0.4385	/
0.0027					
[167/200][0/40]	Loss_D: 1.1178	Loss_G: 8.4378	D(x): 0.9861	D(G(z)): 0.5298	/

0.0013
[168/200][0/40] Loss_D: 0.2475 Loss_G: 3.1384 D(x): 0.8550 D(G(z)): 0.0714 /
0.0771
[169/200][0/40] Loss_D: 1.6709 Loss_G: 8.9670 D(x): 0.9989 D(G(z)): 0.6983 /
0.0005
[170/200][0/40] Loss_D: 1.4893 Loss_G: 9.0031 D(x): 0.9939 D(G(z)): 0.5895 /
0.0033
[171/200][0/40] Loss_D: 4.6361 Loss_G: 9.0804 D(x): 0.9987 D(G(z)): 0.9641 /
0.0016
[172/200][0/40] Loss_D: 1.6473 Loss_G: 8.5934 D(x): 0.9976 D(G(z)): 0.6894 /
0.0008
[173/200][0/40] Loss_D: 0.2011 Loss_G: 4.6071 D(x): 0.9604 D(G(z)): 0.1307 /
0.0186
[174/200][0/40] Loss_D: 0.9759 Loss_G: 7.1841 D(x): 0.9990 D(G(z)): 0.5252 /
0.0017
[175/200][0/40] Loss_D: 0.1888 Loss_G: 4.2100 D(x): 0.9935 D(G(z)): 0.1506 /
0.0227
[176/200][0/40] Loss_D: 0.3593 Loss_G: 5.3007 D(x): 0.9918 D(G(z)): 0.2490 /
0.0079
[177/200][0/40] Loss_D: 0.1081 Loss_G: 4.0475 D(x): 0.9852 D(G(z)): 0.0831 /
0.0318
[178/200][0/40] Loss_D: 0.1364 Loss_G: 4.3539 D(x): 0.8979 D(G(z)): 0.0217 /
0.0287
[179/200][0/40] Loss_D: 2.3243 Loss_G: 10.6474 D(x): 0.9993 D(G(z)): 0.7916 /
0.0002
[180/200][0/40] Loss_D: 0.6704 Loss_G: 7.6891 D(x): 0.9843 D(G(z)): 0.3699 /
0.0017
[181/200][0/40] Loss_D: 2.3745 Loss_G: 10.3325 D(x): 0.9997 D(G(z)): 0.8147 /
0.0002
[182/200][0/40] Loss_D: 4.6447 Loss_G: 9.2554 D(x): 0.9989 D(G(z)): 0.9379 /
0.0065
[183/200][0/40] Loss_D: 0.1668 Loss_G: 3.8994 D(x): 0.9168 D(G(z)): 0.0676 /
0.0368
[184/200][0/40] Loss_D: 0.3931 Loss_G: 5.2465 D(x): 0.9960 D(G(z)): 0.2502 /
0.0110
[185/200][0/40] Loss_D: 0.2141 Loss_G: 4.1319 D(x): 0.9746 D(G(z)): 0.1441 /
0.0297
[186/200][0/40] Loss_D: 0.0342 Loss_G: 4.8865 D(x): 0.9914 D(G(z)): 0.0248 /
0.0146
[187/200][0/40] Loss_D: 2.0917 Loss_G: 8.5923 D(x): 0.9998 D(G(z)): 0.7602 /
0.0009
[188/200][0/40] Loss_D: 0.8768 Loss_G: 7.0877 D(x): 0.9930 D(G(z)): 0.4494 /
0.0019
[189/200][0/40] Loss_D: 0.0827 Loss_G: 4.4033 D(x): 0.9964 D(G(z)): 0.0727 /
0.0186
[190/200][0/40] Loss_D: 0.5021 Loss_G: 5.7880 D(x): 0.9947 D(G(z)): 0.3375 /
0.0050
[191/200][0/40] Loss_D: 0.0891 Loss_G: 4.9574 D(x): 0.9292 D(G(z)): 0.0130 /
0.0139
[192/200][0/40] Loss_D: 0.0329 Loss_G: 4.7567 D(x): 0.9958 D(G(z)): 0.0277 /
0.0153
[193/200][0/40] Loss_D: 0.1173 Loss_G: 4.9196 D(x): 0.8988 D(G(z)): 0.0062 /
0.0150
[194/200][0/40] Loss_D: 0.1024 Loss_G: 3.6386 D(x): 0.9693 D(G(z)): 0.0636 /
0.0439
[195/200][0/40] Loss_D: 0.0386 Loss_G: 4.7405 D(x): 0.9822 D(G(z)): 0.0199 /

```

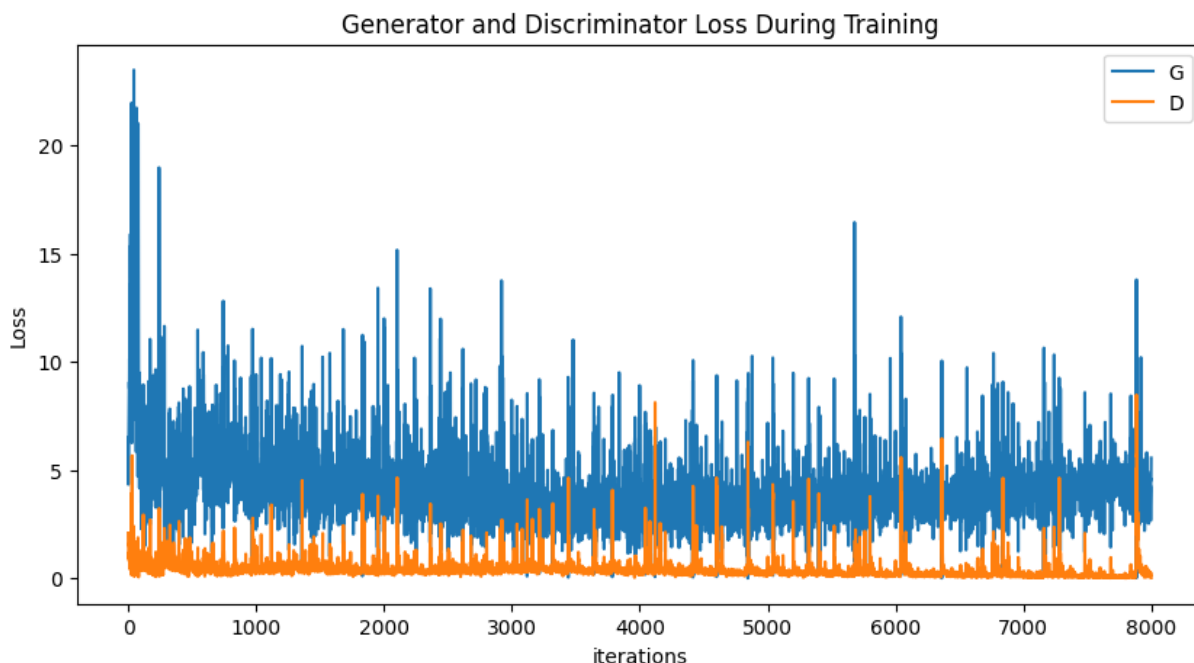
0.0162
[196/200][0/40] Loss_D: 0.6422 Loss_G: 8.4355 D(x): 0.9981 D(G(z)): 0.3750 /
0.0006
[197/200][0/40] Loss_D: 1.2693 Loss_G: 11.0732 D(x): 0.9930 D(G(z)): 0.5774 /
0.0000
[198/200][0/40] Loss_D: 0.4561 Loss_G: 3.9394 D(x): 0.7513 D(G(z)): 0.0638 /
0.0549
[199/200][0/40] Loss_D: 0.4603 Loss_G: 4.8611 D(x): 0.9861 D(G(z)): 0.2831 /
0.0147

```

```

In [61]: plt.figure(figsize=(10,5))
plt.title("Generator and Discriminator Loss During Training")
plt.plot(G_losses,label="G")
plt.plot(D_losses,label="D")
plt.xlabel("iterations")
plt.ylabel("Loss")
plt.legend()
plt.show()

```



```

In [62]: with torch.no_grad():
noise = torch.randn(256, nz, 1, 1, device=device)
fake_images = netG(noise).detach().cpu()

```

```

In [64]: import torchvision.utils as vutils
import os

# Generate fake images if not already done
with torch.no_grad():
    noise = torch.randn(256, nz, 1, 1, device=device) # or any number
    fake_images = netG(noise).detach().cpu()

# Create directory if it doesn't exist
output_dir = "C:/Users/Adarsh/OneDrive/Desktop/cv case study 2/generated_images"
os.makedirs(output_dir, exist_ok=True)

```

```
# Save all generated images
for i in range(fake_images.size(0)):
    save_path = os.path.join(output_dir, f"dcgan_generated_{i+1}.png")
    vutils.save_image(fake_images[i], save_path, normalize=True)

print(f"Saved {fake_images.size(0)} images to: {output_dir}")
```

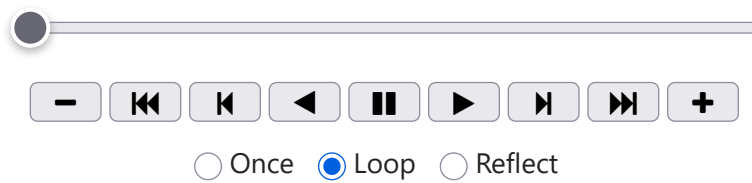
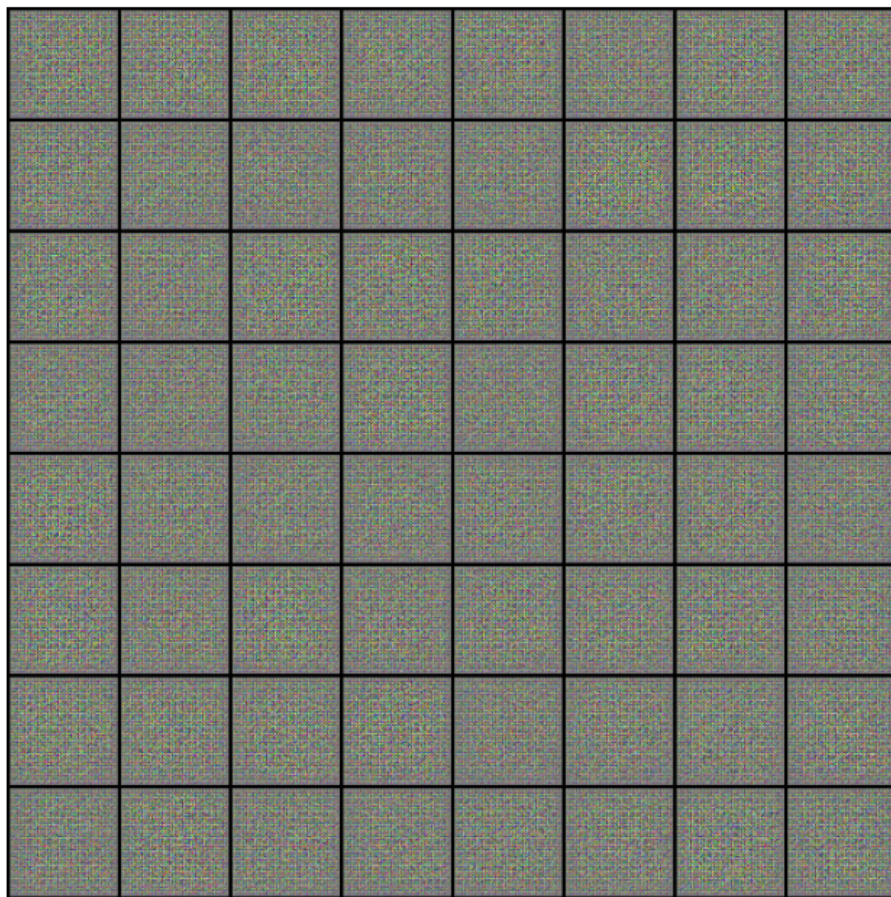
Saved 256 images to: C:/Users/Adarsh/OneDrive/Desktop/cv case study 2/generated_images

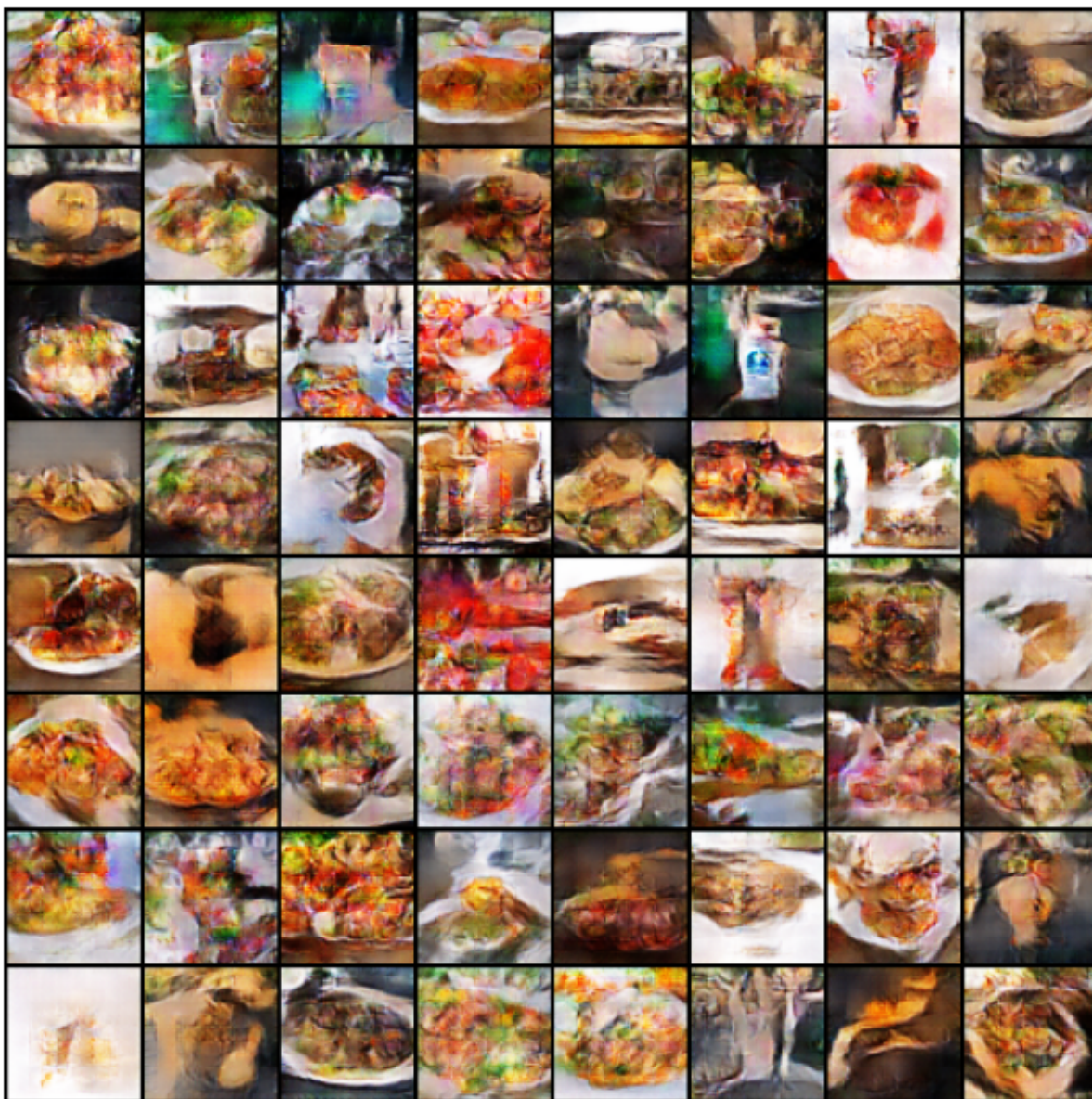
```
In [65]: fig = plt.figure(figsize=(8,8))
plt.axis("off")
ims = [[plt.imshow(np.transpose(i,(1,2,0))), animated=True] for i in img_list]
ani = animation.ArtistAnimation(fig, ims, interval=1000, repeat_delay=1000, blit=True)

HTML(ani.to_jshtml())
```

Animation size has reached 20979489 bytes, exceeding the limit of 20971520.0. If you're sure you want a larger animation embedded, set the animation.embed_limit rc parameter to a larger value (in MB). This and further frames will be dropped.

Out[65]:





```
In [66]: # Grab a batch of real images from the dataloader
real_batch = next(iter(dataloader))

# Plot the real images
plt.figure(figsize=(15,15))
plt.subplot(1,2,1)
plt.axis("off")
plt.title("Real Images")
plt.imshow(np.transpose(vutils.make_grid(real_batch[0].to(device)[:64], padding=5,

# Plot the fake images from the last epoch
plt.subplot(1,2,2)
plt.axis("off")
plt.title("Fake Images")
plt.imshow(np.transpose(img_list[-1],(1,2,0)))
plt.show()
```



```
c:\Users\Adarsh\OneDrive\Desktop\cv case study 2\.venv\Lib\site-packages\torch\utils
\data\data_loader.py:665: UserWarning: 'pin_memory' argument is set as true but no ac
celerator is found, then device pinned memory won't be used.
warnings.warn(warn_msg)
```



```
In [67]: # Comprehensive DCGAN Evaluation Suite - FIXED VERSION
print("=== DCGAN Comprehensive Evaluation (Fixed) ===")

import torch
import torch.nn.functional as F
import torchvision.models as models
from scipy.stats import entropy
from scipy.linalg import sqrtm
import numpy as np
import matplotlib.pyplot as plt

# Load pre-trained InceptionV3 model for evaluation
print("Loading InceptionV3 model...")
inception = models.inception_v3(pretrained=True, transform_input=False)
inception.eval()
inception = inception.to(device)

# Create proper feature extractor that stops BEFORE the final avgpool and fc layers
# We want to extract features from the last convolutional layers (Mixed_7c)
class InceptionFeatureExtractor(torch.nn.Module):
    def __init__(self, inception_model):
        super().__init__()
        # Take everything up to Mixed_7c (the last convolutional block)
        self.features = torch.nn.Sequential(
            inception_model.Conv2d_1a_3x3,
            inception_model.Conv2d_2a_3x3,
            inception_model.Conv2d_2b_3x3,
            torch.nn.MaxPool2d(3, stride=2),
            inception_model.Conv2d_3b_1x1,
            inception_model.Conv2d_4a_3x3,
            torch.nn.MaxPool2d(3, stride=2),
            inception_model.Mixed_5b,
            inception_model.Mixed_5c,
```

```

        inception_model.Mixed_5d,
        inception_model.Mixed_6a,
        inception_model.Mixed_6b,
        inception_model.Mixed_6c,
        inception_model.Mixed_6d,
        inception_model.Mixed_6e,
        inception_model.Mixed_7a,
        inception_model.Mixed_7b,
        inception_model.Mixed_7c,
    )

    def forward(self, x):
        return self.features(x)

feature_extractor = InceptionFeatureExtractor(inception)
feature_extractor.eval()

print("Fixed evaluation models loaded successfully!")

```

```

=== DCGAN Comprehensive Evaluation (Fixed) ===
Loading InceptionV3 model...

```

```

c:\Users\Adarsh\OneDrive\Desktop\cv case study 2\.venv\Lib\site-packages\torchvision\models\_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.

```

```

    warnings.warn(
c:\Users\Adarsh\OneDrive\Desktop\cv case study 2\.venv\Lib\site-packages\torchvision\models\_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=Inception_V3_Weights.IMAGENET1K_V1`. You can also use `weights=Inception_V3_Weights.DEFAULT` to get the most up-to-date weights.

```

```

    warnings.warn(msg)
Fixed evaluation models loaded successfully!

```

```

In [68]: def calculate_inception_score(images, inception_model, splits=10, batch_size=32):
        """
        Calculate Inception Score for generated images
        """
        print(f"Calculating Inception Score for {images.size(0)} images...")

        N = images.size(0)
        assert N >= splits, f"Number of images ({N}) must be at least equal to splits ("

        # Resize images to 299x299 for InceptionV3
        images_resized = F.interpolate(images, size=(299, 299), mode='bilinear', align_
        # Normalize to [0, 1] for InceptionV3
        images_resized = (images_resized + 1) / 2

        preds = []
        for i in range(0, N, batch_size):
            batch = images_resized[i:i+batch_size]
            with torch.no_grad():
                pred = inception_model(batch)
                if isinstance(pred, tuple): # InceptionV3 returns (logits, aux_logits)
                    pred = pred[0]
                pred = F.softmax(pred, dim=1).cpu().numpy()

```

```

        preds.append(pred)

preds = np.concatenate(preds, axis=0)

# Calculate IS for each split
split_scores = []
for k in range(splits):
    part = preds[k * (N // splits): (k + 1) * (N // splits), :]
    py = np.mean(part, axis=0)
    scores = []
    for i in range(part.shape[0]):
        pyx = part[i]
        scores.append(entropy(pyx, py))
    split_scores.append(np.exp(np.mean(scores)))

return float(np.mean(split_scores)), float(np.std(split_scores))

# Test Inception Score
try:
    mean_is, std_is = calculate_inception_score(fake_images, inception, splits=10)
    print(f"✓ Inception Score: {mean_is:.4f} ± {std_is:.4f}")
except Exception as e:
    print(f"✗ Inception Score calculation failed: {e}")
    mean_is, std_is = None, None

```

Calculating Inception Score for 256 images...

✓ Inception Score: 2.7074 ± 0.3253

```

In [ ]: def calculate_fid_score_fixed(real_images, fake_images, feature_extractor, batch_si
        """
        Calculate FID (Frechet Inception Distance) score using PROPER feature extractio
        """
        print(f"Calculating FID for {real_images.size(0)} real and {fake_images.size(0)}

        def preprocess_images(images):
            # Resize to 299x299 for InceptionV3
            images_resized = F.interpolate(images, size=(299, 299), mode='bilinear', al
            # Normalize to [0, 1] for InceptionV3
            images_resized = (images_resized + 1) / 2
            return images_resized

        def get_activations(images, model):
            activations = []
            images = preprocess_images(images)

            for i in range(0, images.size(0), batch_size):
                batch = images[i:i+batch_size]
                with torch.no_grad():
                    # Get features from the feature extractor (convolutional features)
                    features = model(batch) # This will be [batch_size, 2048, 8, 8]
                    # Apply global average pooling to get [batch_size, 2048]
                    features = F.adaptive_avg_pool2d(features, (1, 1))
                    features = features.view(features.size(0), -1) # Flatten to [batch
                    activations.append(features.cpu().numpy())

            return np.concatenate(activations, axis=0)

```

```

# Get activations for real and fake images
act_real = get_activations(real_images, feature_extractor)
act_fake = get_activations(fake_images, feature_extractor)

print(f"Real activations shape: {act_real.shape}")
print(f"Fake activations shape: {act_fake.shape}")

# Calculate statistics
mu_real, sigma_real = act_real.mean(axis=0), np.cov(act_real, rowvar=False)
mu_fake, sigma_fake = act_fake.mean(axis=0), np.cov(act_fake, rowvar=False)

# Calculate FID
ssdiff = np.sum((mu_real - mu_fake) ** 2)
covmean = sqrtm(sigma_real.dot(sigma_fake))

# Numerical stability
if np.iscomplexobj(covmean):
    covmean = covmean.real

fid = ssdiff + np.trace(sigma_real + sigma_fake - 2 * covmean)
return float(fid)

# Test Fixed FID Score
try:

    real_imgs = real_batch[0][:256].to(device)
    fake_imgs = fake_images[:256].to(device)
    fid_score = calculate_fid_score_fixed(real_imgs, fake_imgs, feature_extractor)
    print(f"✓ FID Score: {fid_score:.4f}")
except Exception as e:
    print(f"X FID calculation failed: {e}")
    import traceback
    traceback.print_exc()
    fid_score = None

```

Calculating FID for 128 real and 256 fake images...

Real activations shape: (128, 2048)

Fake activations shape: (256, 2048)

✓ FID Score: 205.9182

```

In [71]: def calculate_diversity_metrics(images):
    """
    Calculate diversity metrics for generated images
    """
    print("Calculating diversity metrics...")

    # Convert to numpy and flatten
    images_np = images.cpu().numpy()
    images_flat = images_np.reshape(images_np.shape[0], -1)

    # Calculate pairwise distances
    from sklearn.metrics.pairwise import cosine_distances, euclidean_distances

    cosine_dist = cosine_distances(images_flat)
    euclidean_dist = euclidean_distances(images_flat)

```

```

# Get upper triangle (excluding diagonal)
triu_indices = np.triu_indices(len(images_flat), k=1)
cosine_diversity = cosine_dist[triu_indices].mean()
euclidean_diversity = euclidean_dist[triu_indices].mean()

return {
    'cosine_diversity': float(cosine_diversity),
    'euclidean_diversity': float(euclidean_diversity),
    'pairwise_comparisons': len(triu_indices[0])
}

def calculate_image_statistics(images):
    """
    Calculate basic image statistics
    """
    print("Calculating image statistics...")

    images_np = images.cpu().numpy()

    stats = {
        'mean_pixel_value': float(images_np.mean()),
        'std_pixel_value': float(images_np.std()),
        'min_pixel_value': float(images_np.min()),
        'max_pixel_value': float(images_np.max()),
        'mean_channel_variance': float(np.var(images_np, axis=(2, 3)).mean())
    }

    return stats

def analyze_training_stability():
    """
    Analyze training stability from loss curves
    """
    print("Analyzing training stability...")

    g_losses_np = np.array(G_losses)
    d_losses_np = np.array(D_losses)

    # Calculate rolling statistics
    window = 100
    if len(g_losses_np) > window:
        g_rolling_std = np.convolve(g_losses_np, np.ones(window)/window, mode='valid')
        d_rolling_std = np.convolve(d_losses_np, np.ones(window)/window, mode='valid')

        stability_metrics = {
            'g_loss_final_std': float(np.std(g_losses_np[-window:])),
            'd_loss_final_std': float(np.std(d_losses_np[-window:])),
            'g_loss_mean': float(np.mean(g_losses_np)),
            'd_loss_mean': float(np.mean(d_losses_np)),
            'loss_ratio': float(np.mean(g_losses_np) / np.mean(d_losses_np))
        }
    else:
        stability_metrics = {
            'g_loss_std': float(np.std(g_losses_np)),
            'd_loss_std': float(np.std(d_losses_np)),

```

```

        'g_loss_mean': float(np.mean(g_losses_np)),
        'd_loss_mean': float(np.mean(d_losses_np)),
        'loss_ratio': float(np.mean(g_losses_np) / np.mean(d_losses_np))
    }

    return stability_metrics

# Calculate additional metrics
try:
    diversity_metrics = calculate_diversity_metrics(fake_images)
    print(f"✓ Diversity metrics calculated")
except Exception as e:
    print(f"✗ Diversity metrics failed: {e}")
    diversity_metrics = {}

try:
    image_stats = calculate_image_statistics(fake_images)
    print(f"✓ Image statistics calculated")
except Exception as e:
    print(f"✗ Image statistics failed: {e}")
    image_stats = {}

try:
    stability_metrics = analyze_training_stability()
    print(f"✓ Training stability analyzed")
except Exception as e:
    print(f"✗ Training stability analysis failed: {e}")
    stability_metrics = {}

```

Calculating diversity metrics...

✓ Diversity metrics calculated

Calculating image statistics...

✓ Image statistics calculated

Analyzing training stability...

✓ Training stability analyzed

```

In [72]: # Comprehensive Evaluation Summary
print("\n" + "="*60)
print("          DCGAN COMPREHENSIVE EVALUATION RESULTS")
print("="*60)

# Core Metrics
print("\n🎯 CORE GENERATION METRICS:")
if mean_is is not None:
    print(f"    Inception Score: {mean_is:.4f} ± {std_is:.4f}")
    interpretation = "Excellent" if mean_is > 8 else "Good" if mean_is > 5 else "Fair"
    print(f"    → Interpretation: {interpretation} (Higher is better)")
else:
    print("    Inception Score: Failed to calculate")

if fid_score is not None:
    print(f"    FID Score: {fid_score:.4f}")
    interpretation = "Excellent" if fid_score < 50 else "Good" if fid_score < 100 else "Fair"
    print(f"    → Interpretation: {interpretation} (Lower is better)")
else:
    print("    FID Score: Failed to calculate")

```



```

# Diversity Metrics
print("\n🌈 DIVERSITY METRICS:")
if diversity_metrics:
    print(f"    Cosine Diversity: {diversity_metrics['cosine_diversity']:.4f}")
    print(f"    Euclidean Diversity: {diversity_metrics['euclidean_diversity']:.4f}")
    print(f"    → Higher values indicate more diverse generated images")

# Image Quality Metrics
print("\n🖼️ IMAGE QUALITY METRICS:")
if image_stats:
    print(f"    Mean Pixel Value: {image_stats['mean_pixel_value']:.4f}")
    print(f"    Pixel Std Dev: {image_stats['std_pixel_value']:.4f}")
    print(f"    Pixel Range: [{image_stats['min_pixel_value']:.4f}, {image_stats['ma
    print(f"    Channel Variance: {image_stats['mean_channel_variance']:.4f}")

# Training Stability
print("\n📊 TRAINING STABILITY METRICS:")
if stability_metrics:
    print(f"    Generator Loss Mean: {stability_metrics['g_loss_mean']:.4f}")
    print(f"    Discriminator Loss Mean: {stability_metrics['d_loss_mean']:.4f}")
    print(f"    Loss Ratio (G/D): {stability_metrics['loss_ratio']:.4f}")

    balance_interpretation = "Well balanced" if 0.5 < stability_metrics['loss_ratio
    print(f"    → Training Balance: {balance_interpretation}")

# Model Configuration Summary
print("\n⚙️ MODEL CONFIGURATION:")
print(f"    Training Epochs: {num_epochs}")
print(f"    Batch Size: {batch_size}")
print(f"    Learning Rate: {lr}")
print(f"    Image Size: {image_size}x{image_size}")
print(f"    Latent Dimension: {nz}")
print(f"    Training Samples: {len(dataset)}")

# Overall Assessment
print("\n🏆 OVERALL ASSESSMENT:")
overall_score = 0
if mean_is is not None and mean_is > 3:
    overall_score += 1
if fid_score is not None and fid_score < 200:
    overall_score += 1
if stability_metrics and 0.5 < stability_metrics.get('loss_ratio', 0) < 2.0:
    overall_score += 1

assessment = ["Poor", "Fair", "Good", "Excellent"][overall_score]
print(f"    Model Performance: {assessment} ({overall_score}/3 criteria met)")

# Recommendations
print("\n💡 RECOMMENDATIONS:")
if mean_is is None or fid_score is None:
    print("    - Fix evaluation metric calculations")
if mean_is is not None and mean_is < 5:
    print("    - Consider increasing model capacity or training longer")
if fid_score is not None and fid_score > 100:
    print("    - Improve image quality through architectural changes")

```



```
if stability_metrics and stability_metrics.get('loss_ratio', 1) > 2:
    print("    - Balance training by adjusting learning rates")

print("\n" + "="*60)
print("Evaluation completed! Generated images saved in:", output_dir)
print("="*60)
```

=====

DCGAN COMPREHENSIVE EVALUATION RESULTS

=====



CORE GENERATION METRICS:

Inception Score: 2.7074 ± 0.3253
→ Interpretation: Poor (Higher is better)
FID Score: 205.9182
→ Interpretation: Poor (Lower is better)



DIVERSITY METRICS:

Cosine Diversity: 0.8618
Euclidean Diversity: 79.1519
→ Higher values indicate more diverse generated images



IMAGE QUALITY METRICS:

Mean Pixel Value: -0.0250
Pixel Std Dev: 0.5510
Pixel Range: [-0.9996, 1.0000]
Channel Variance: 0.2026



TRAINING STABILITY METRICS:

Generator Loss Mean: 4.1745
Discriminator Loss Mean: 0.4276
Loss Ratio (G/D): 9.7625
→ Training Balance: Imbalanced



MODEL CONFIGURATION:

Training Epochs: 200
Batch Size: 128
Learning Rate: 0.0002
Image Size: 64x64
Latent Dimension: 100
Training Samples: 5000



OVERALL ASSESSMENT:

Model Performance: Poor (0/3 criteria met)



RECOMMENDATIONS:

- Consider increasing model capacity or training longer
- Improve image quality through architectural changes
- Balance training by adjusting learning rates

=====

Evaluation completed! Generated images saved in: C:/Users/Adarsh/OneDrive/Desktop/cv case study 2/generated_images

=====

After this iteration the IS score is increased which is a good news and FID also reduced so we

are back on track.